Calango

User Guide

Visual atomistic modeling,
simulation and analysis

Leandro Seixas

Version 26.8.11

About this guide

Calango is a desktop application for building, visualizing, simulating and analysing atomic structures. It pairs a C++20 / OpenGL core — fast enough to orbit tens of thousands of atoms in real time — with an embedded CPython interpreter running the Atomic Simulation Environment (ASE), so every structure you see on screen can be handed to a real calculator without leaving the program.

This guide is written for the person using Calango, not the person building it. It walks through the workspace, then covers each menu in the order you are likely to need it: getting data in, building and editing structures, making them look right, running simulations locally or on a cluster, and analysing the results. Chapter 16 is a reference you can jump straight to — keyboard shortcuts, the complete menu map, file formats and troubleshooting.

Conventions

Analysis ⟩ Raman Modes A menu path. Each ⟩ step is one level deeper.

Compute A button, field, checkbox or tab label exactly as it appears in the interface.

Ctrl+R A keyboard shortcut. On macOS, Ctrl means Cmd wherever the shortcut is a standard editing action (open, save, quit, undo); single-letter viewport shortcuts have no modifier on any platform.

`run.py` A file, directory or path.

Three kinds of callout appear throughout:

> **Note**
> Background you do not strictly need, but that explains why something behaves the way it does.

> **Tip**
> A shortcut or working habit that saves time.

> **Requires**
> An external dependency — a Python package, a solver binary, a network connection — that the feature cannot work without.

Version

This guide documents Calango **26.8.11**. The version you are running is shown in **Help ⟩ About Calango**, together with the Python and ASE versions actually in use — worth checking first whenever something behaves unexpectedly.

Contents

CHAPTER 1

Getting Started

1.1 Installing Calango

Calango ships as a native installer for each desktop platform:

macOS A drag-and-drop `.dmg`. Mount it and drag `calango.app` onto the `Applications` shortcut.

Linux A Debian package. Install it with `apt` so dependencies resolve automatically:

```
sudo apt install ./calango_26.8.2_amd64.deb
```

The package registers a desktop launcher, an application icon, and a MIME type for `.calproj` files, so double-clicking a project in your file manager opens the workspace.

Building from source is covered in the companion *Calango Packaging Guide* (`docs/packaging.pdf`), which also documents how the installers are produced.

1.2 The Python environment

Calango embeds a CPython interpreter and drives ASE through it. Almost everything that touches chemistry — reading a CIF, building a slab, running a calculation — goes through that interpreter, so pointing Calango at a Python that has ASE installed is the single most important piece of setup.

1.2.1 How the interpreter is chosen

An embedded interpreter does not inherit an activated virtual environment the way a shell does, so Calango resolves one explicitly, in this order:

1. `$CALANGO_PYTHON` — an explicit path to a Python executable. Highest priority; use this to override everything else.
2. `$VIRTUAL_ENV` — an activated virtual environment, if Calango was launched from that shell.
3. A Python bundled inside the installed application (`Contents/Resources/python` on macOS, `<prefix>/lib/calango/python` on Linux), when the installer was built with one.
4. The interpreter the build was configured against.

Requires

Calango needs `ase` and `numpy` at minimum. Individual features add their own requirements: `spglib` (symmetry, Raman modes), `paramiko` (remote access), `pillow` / `imageio` / `imageio-ffmpeg` (animation export), `mace-torch` (ML potentials), `gpaw` (DFT bands and PDOS). A complete environment:

```
python3 -m venv .venv
.venv/bin/pip install ase numpy spglib paramiko pillow imageio \
                      imageio-ffmpeg
```

1.2.2 Checking the environment

Before opening any structure, confirm the interpreter is the one you expect:

```
calango --probe-python
```

This runs headlessly and prints the resolved interpreter, the Python version and the ASE version, exiting with status 0 only when ASE imports successfully. It is the first thing to run when structure loading fails.

If ASE cannot be imported at launch, Calango still starts — so you can adjust settings — but structure I/O and job features are disabled, and a warning explains how to point at a working interpreter.

Tip

Simulation jobs do not have to use the embedded interpreter. The calculator, phonon and band-structure dialogs each carry an **Execution Environment** selector that routes the job to any Conda environment or interpreter path (Section 9.3). This is how you keep, say, a GPAW environment separate from the one Calango itself uses.

1.2.3 The environment file

At launch Calango reads an environment file — `~/.env` by default — and exports `MP_API_KEY`, the Materials Project API key, into the process environment. A key already present in your shell takes precedence at startup.

Point Calango at a different file, or re-read it on demand, in **Edit ⟩ Preferences**. The dialog reports whether the file exists and whether it actually defines a key. The same controls are mirrored in the **Materials Project** tab of the database browser.

1.3 First launch

Calango opens with an empty workspace. Three quick ways to get something on screen:

1. **File ⟩ Open ⟩ Structure** and pick any structure file (Section 5.1 lists the formats).
2. **Build ⟩ From Database** and load one of the bundled presets — diamond, bulk and monolayer MoS_2, graphene, benzene, naphthalene, coronene.
3. Pass a file on the command line: `calango structure.cif`. Each file argument opens in its own tab; a `.calproj` argument restores a whole saved session.

CHAPTER 2

Tutorial: silicon from start to finish

This tutorial walks the whole arc of a materials study on one small, well-understood system: crystalline silicon. You will build the crystal, relax it, converge a ground state, compute the phonon dispersion, and finish with the electronic band structure and projected density of states.

Silicon is the right first system precisely because the answers are known. Every stage below ends with a number you can check, so when something goes wrong you find out at that stage instead of three stages later.

> **Requires**
> GPAW is used throughout. Install it into a Conda environment and point Calango at it once, in **Edit** ⟩ **Preferences** ⟩ **Python & Environments**. Every wizard then binds to that environment automatically. Without GPAW you can still follow the tutorial with the **EMT** calculator — the workflow is identical — except for the absence of electronic structure calculations.

2.1 Step 1 — Build the crystal

Open **Build** ⟩ **From Database...** and stay on the **Bulk** tab. It is already set up for this system:

Build from `Prototype (ase.build.bulk)`.

Formula `Si`.

Crystal structure `diamond`.

Lattice constant a leave at `0` to accept ASE's tabulated value, 5.43 Å.

Cell shape leave unchecked for the two-atom primitive cell.

Click **Build Crystal** and the crystal opens in a new workspace tab. The **Structure** dock on the left should now read `Si2`, 2 atoms, with $a = b = c = 3.84$ Å and $\alpha = \beta = \gamma = 60°$: that is the rhombohedral primitive cell of the diamond lattice, not an error. The conventional cubic cell with its 8 atoms and 90° angles is what you get by ticking **Conventional cubic cell**.

> **Tip**
> Work in the primitive cell. Every stage after this one scales with the number of atoms — the phonon step worst of all, since it builds a supercell of whatever you hand it. Eight atoms instead of two would make that step roughly an order of magnitude slower for identical physics.

2.2 Step 2 — Relax the geometry

Silicon's tabulated lattice constant is not PBE's lattice constant, so the first job is to let the calculator find its own minimum.

Open **Simulation** ⟩ **Geometry Optimization...**. The wizard runs in four stages; the **Next** button carries you through them.

1. **Calculator & Execution Environment.** Choose **GPAW**. The Conda environment is filled in from Preferences.
2. **Calculator Settings.** For this system:
   - **Mode** `PW`, **Plane-wave cutoff** `400 eV`
   - **XC functional** `PBE`
   - **k-points** `8 × 8 × 8`
3. **Optimization Settings. Force convergence (fmax)** `0.01 eV/Å`. Silicon's primitive cell has no free internal coordinates by symmetry, so this converges almost immediately.
4. **ASE Script Review.** Read the script — it is the whole job, and it is editable. Press **Run**.

The **Processes** dock tracks the run and the **Results** dock plots energy against step as it goes. When it finishes, the relaxed structure loads back into the tab and **Results** ⟩ **Geometry Optimization Viewer...** shows the energy and force convergence history.

> **Note**
> Relaxing *positions* will not change silicon's lattice constant — BFGS moves atoms, not the cell. Getting PBE's lattice constant (about 5.47 Å, some 0.7 % above experiment, which is PBE behaving normally) means an energy–volume scan: build several cells with **Build** ⟩ **Supercell...** or the structure editor, run a single point on each, and fit the minimum. This tutorial keeps the tabulated constant so the later stages stay comparable to published numbers.

2.3 Step 3 — Converge the ground state

Simulation ⟩ **Single-point Calculation...** (Ctrl+R) now does double duty. It reports the total energy, and — because the calculator is GPAW — it writes `single_point.gpw`, the converged density and wavefunctions.

Use the same calculator settings as Step 2, but raise the k-grid to `12 × 12 × 12`. Run it.

> **Note**
> This file is the *baseline* that the later analysis workflows inherit. Electronic Structure, Optics, Wannier Functions, GW and the charge density difference all load a completed ground state and evaluate at fixed density rather than re-converging their own. That is deliberate: a spectrum computed from a silently different SCF solution than the one you inspected is not attributable to anything. It also means those wizards will refuse to open

until this step exists — if one tells you it needs a baseline, this is the step you skipped.

Results ⟩ Single-Point Viewer... shows the total energy, the Fermi level and the maximum residual force.

2.4 Step 4 — Phonon dispersion

Open **Simulation ⟩ Phonon...**. This wizard has four stages.

1. **Calculator & Execution Environment.** GPAW again.
2. **Calculator Settings.** As before.
3. **Phonon Settings.**
   - **Displacement** δ `0.01` Å — small enough to stay harmonic, large enough to lift the forces clear of numerical noise.
   - **Supercell** `3 × 3 × 3`. The supercell sets how far force constants are sampled, and therefore how much of the dispersion is real rather than interpolated. $2 \times 2 \times 2$ runs faster and already gives the right shape.
   - **Symmetry-reduced displacements on**. The naive scheme costs $6N$ force evaluations; silicon's space group relates almost all of them, and spglib (through phonopy) reduces the set by roughly an order of magnitude for identical force constants.
   - **Remove residual forces on**. A relaxation stops at a finite `fmax`, so the reference geometry still carries small forces; left in, they add a spurious linear term that shows up as non-zero acoustic frequencies at Γ.
   - **Enforce acoustic sum rule on**.
4. **q-Path Definition.** The embedded Brillouin-zone builder suggests silicon's conventional path, $\Gamma \to X \to W \to K \to \Gamma \to L$. Accept it.

This is the longest step of the tutorial — it is many force evaluations on a supercell. When it completes, the **Phonon Viewer** opens on its own.

2.4.1 Checking the answer

Two atoms in the primitive cell give $3 \times 2 = 6$ branches: three acoustic and three optical. Check these before reading anything else into the plot:

Acoustic branches vanish at Γ. All three should start at zero. Residual frequencies of a few cm^{-1} are normal; tens of cm^{-1} mean the acoustic sum rule or the residual-force subtraction is not doing its job, or the relaxation was too loose.

The optical mode at Γ sits near 15.5 THz (520 cm^{-1}). This is silicon's Raman line, one of the best-measured numbers in solid state physics. Within a few percent means the calculation is sound.

No imaginary frequencies. Plotted as negative, they would mean the structure is not at a minimum — for silicon that indicates a problem with the setup, not a real instability.

Analysis ⟩ **Phonon Thermodynamics...** integrates the density of states into internal energy, free energy, entropy and heat capacity over 0–1000 K. Two checks come free: C_V must approach the Dulong–Petit limit $3Nk_B$ at high temperature, and the entropy must go to zero at 0 K.

2.5 Step 5 — Electronic structure

Open **Simulation** ⟩ **Electronic Structure...**. Stage 1 asks for a **Baseline SCF density** — the `single_point.gpw` from Step 3. Select it.

The run is non-self-consistent by construction: it loads that density and evaluates the bands along the k-path at fixed density. Set the k-path the same way as for the phonons ($\Gamma \to X \to W \to K \to \Gamma \to L$) and enable **PDOS** to get the element- and orbital-projected density of states beside the bands.

When it finishes, the band/PDOS viewer opens: bands as $E - E_F$ against k-distance on the left, the PDOS sharing the energy axis on the right.

2.5.1 Checking the answer

The gap is indirect. The valence-band maximum sits at Γ; the conduction-band minimum does not — it lies along $\Gamma \to X$, about 85 % of the way to X. A band structure showing silicon's minimum at Γ is wrong.

The gap is too small, and that is expected. PBE gives roughly 0.6 eV against an experimental 1.17 eV. This is the standard band-gap underestimation of semilocal DFT, not a convergence failure. Raising the cutoff or the k-grid will not fix it, because it is not an error in the calculation.

The PDOS is sp^3. The valence band is a mixture of Si s and p character, as tetrahedral bonding requires.

> **Tip**
> To correct the gap rather than accept it, run **Simulation** ⟩ **GW Calculations...** on the same baseline. One-shot G_0W_0 replaces the DFT exchange–correlation potential with the self-energy and opens semiconductor gaps by typically 0.5–2 eV. The **GW Viewer** reports the DFT gap, the quasiparticle gap and the renormalization between them.

2.6 Where to go next

The same baseline supports the rest of the analysis suite without recomputing the ground state:

- **Simulation** ⟩ **Optics...** — the dielectric function $\varepsilon(\omega)$ and the absorption, reflectivity, refractive-index and loss spectra derived from it.
- The **ELF** checkbox under **Density Exports** in the single-point setup — the covalent bond charge between neighbours, written as `elf.cube` by the same SCF and rendered as an isosurface in the **Volumetric Data** dock. There is no separate ELF module: the field comes out of the ground state you already have.
- **Analysis** ⟩ **Charge Density Difference (CDD)...** — $\Delta\rho = \rho_{A+B} - \rho_A - \rho_B$ between two

fragments of the same run, showing where the charge went when they were put together.

- **Simulation ⟩ Wannier Functions…** — the four sp^3 bond orbitals of the diamond lattice.
- **Analysis ⟩ Raman Modes…** — the factor-group analysis that labels silicon's Γ optical mode T_{2g} and Raman-active.

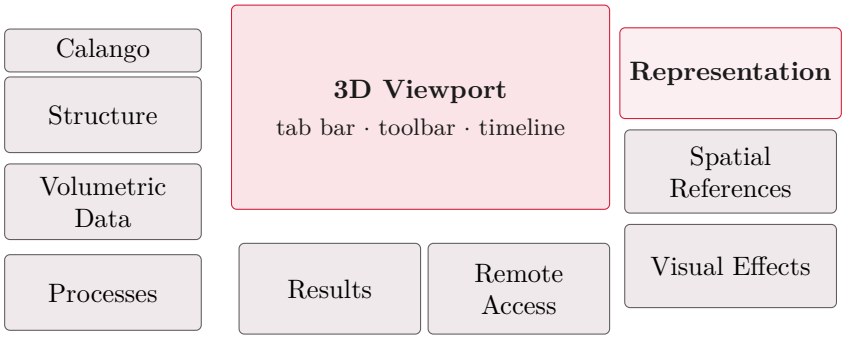
Save the workspace with **File ⟩ Save** before you stop. Job directories of a saved project are kept alongside the `.calproj`, so every result above stays linked in the **Processes** dock and can be reopened without recomputation.

CHAPTER 3

The Workspace

3.1 The default layout

Calango opens into two full-height side columns flanking the 3D viewport, with a short row of job panels between them. Every panel is a Qt dock widget, so the layout is a starting point rather than a constraint — any panel can be resized, re-docked, tabbed with another, floated onto a second monitor, or hidden entirely.



Calango The branding strip heading the left column. Purely decorative; hide it from **View** if you want the vertical space.

Structure Formula, atom and bond counts, lattice parameters, cell volume and periodicity, plus **Edit Structure...** (Chapter 7).

Volumetric Data Scalar fields — charge density, ELF, Wannier orbitals — as isosurfaces and colour slices (Section 14.10).

Processes The process manager (Section 3.5): every background task with live status.

the viewport The document tab bar, the frame toolbar, the 3D canvas, and the playback timeline (visible only for trajectories). Covered in Chapter 4.

Results Console and live metric plots for the running calculation (Section 9.4).

HPC SSH connection, cluster job submission and queue monitoring (Chapter 10). It opens at the narrowest width that still shows its whole form.

Representation Appearance controls — style, mode, color mapping, radii and bond widths (Chapter 8).

Spatial References Three tabs: the cell wireframe, the orientation triad, and the per-atom vector overlay. All three draw something onto the scene that is not the atoms.

Under **Vectors**, **Vector scale** and **Vector width** set the arrows' length and thickness. Arrowheads are always drawn — a plain shaft does not say which way a vector points, which is the one thing the overlay exists to state; thinning the arrows is what relieves the clutter on a dense magnetic structure.

Under **Unit cell**, **Show atoms of the neighboring unit cell** draws exactly the periodic images that terminate a bond leaving the cell, so no bond ends in mid-air. Only those: duplicating every atom on a face filled the view with periodic repetition rather than information.

Visual Effects Lighting, shadow, fog, blur and ambient occlusion.

> **Tip**
> Your layout is saved when you quit and restored on the next launch, including floating panels and window geometry. Every panel has a toggle at the bottom of the **View** menu, so a panel you closed by accident is one menu click away — and **View** ⟩ **Reset Layout** puts everything back where this diagram shows it.

> **Note**
> After an upgrade that changes the default arrangement, Calango discards the saved layout once so the new default appears; any rearrangement you make after that persists as usual.

3.2 Documents, tabs and the timeline

Each structure or trajectory you open occupies its own *tab* above the viewport, and each tab carries its own undo history. All tabs share one accelerated viewport — switching tabs changes which document the views observe, so display settings stay consistent and no OpenGL context is recreated.

When a document contains more than one frame, the **playback timeline** appears below the viewport:

Transport buttons first, previous, play/pause, next, last.

Scrubber a tick-marked slider over the frame range.

Rate playback speed in frames per second, 0.1–120, default 15.

Trajectories arrive from several places: opening a multi-frame file, finishing an MD or relaxation run, generating displaced structures in the phonon builder, or producing a noise ensemble. During a running simulation the timeline grows in real time as frames stream in (Section 9.5).

3.3 The Structure panel

The **Structure** panel reports what Calango knows about the active document:

Formula Hill-ordered chemical formula.

Atoms, Bonds Counts, with bonds as currently perceived.

a, b, c Cell vector lengths in Å, to two decimals.

α, β, γ Cell angles in degrees.

Cell volume, Periodic Volume and per-axis periodicity.

Tolerance The spglib symmetry tolerance (*symprec*) used for the three fields below, in Å, up to four decimals. Default 0.001. Raise it for structures with numerical noise — a relaxed cell often needs 0.01 or more before its true symmetry is recognised.

Space group, Point group, Crystal system Detected symmetry, e.g. `Fd-3m (227)`, `m-3m`, `cubic`.

> **Requires**
> Symmetry detection needs `spglib` and a defined unit cell.

3.4 Projects and session storage

A *project* is one `.calproj` file that restores an entire session: every tab with its structures and trajectory frames, the viewport colour mapping, and the job console with its recorded metric series.

File ⟩ **Project Workspace** ⟩ **Open Project** Ctrl+Shift+O. Replaces the current session, warning first if tabs are open.

File ⟩ **Project Workspace** ⟩ **Save Project** Ctrl+S.

File ⟩ **New Workspace** Ctrl+N. Closes everything and starts clean.

Calango tracks unsaved changes. Every undoable edit, tab close and job launch marks the session dirty; saving or loading a project clears the flag. Quitting with unsaved changes asks whether to **Save**, **Discard** or **Cancel** — cancelling (or cancelling the save dialog that follows) leaves the application open.

3.4.1 The managed temporary directory

Simulation jobs and generated ensembles need somewhere to put their working files. Once a project has been saved, Calango places them in a `.calango_tmp/` folder *next to the `.calproj` file*, one timestamped subdirectory per task:

```
my_study.calproj
.calango_tmp/
    job_20260721_143255/     run.py, structure.extxyz, md.traj, logs
    noise_20260721_144012/   perturbed.extxyz
    job_20260721_150130/     bands.json, pdos.json
```

Until a project has been saved, the same directories are created under the per-user application data location instead. Either way the process manager keeps a live link to each one.

3.5 The process manager

The compact **Processes** panel in zone 5 lists every background task of the session — local calculations, remote submissions, band-structure runs, noise ensembles — with its name, colour-coded status (*queued*, *running*, *completed*, *failed*) and start time.

Two buttons act on the selected task:

Open Folder Reveals the task's working directory in your file manager, for the raw logs and any files Calango does not import.

Load Result Brings the result back into the workspace. Double-clicking the row does the same. Calango inspects the directory and opens whatever it finds: band structure and PDOS data open in the electronic-structure viewer, trajectories and final geometries open as tabs.

The point of keeping these links is that a finished calculation stays one click from further analysis. A completed MD run can be re-loaded weeks later and fed to the RDF, distribution or dataset tools without recomputing anything.

CHAPTER 4

The 3D Viewport

The viewport is an OpenGL 3.3 core-profile canvas using instanced rendering — one draw call per mesh type — so structures with tens of thousands of atoms stay interactive.

4.1 Interaction modes

What a left-drag does depends on the active *interaction mode*. The six modes are the first group of buttons on the frame toolbar, each with a single-letter shortcut:

Key	Mode	Left mouse button
R	Rotation	Drag orbits the camera around the structure. The default.
T	Translation	Drag pans the scene.
S	Selection	Drag draws a rubber-band box and selects every atom inside it.
I	Insertion	Click empty space to add an atom; drag from one atom to another to bond them.
D	Distance	Click two atoms to measure their separation.
A	Angle	Click three atoms (vertex second) to measure the angle.

Three gestures work in *every* mode, so navigation never requires switching back:

Middle-drag, or Shift+left-drag Pan.

Wheel Zoom.

Double-click Reframe the structure.

The mouse cursor changes with the mode — an arrow for rotation, an open hand for panning, a crosshair for selection, insertion and measurement.

4.2 Selecting atoms

A left *click* (no drag) picks the atom under the cursor by ray-cast, in any mode. Ctrl+click (Cmd+click on macOS) toggles an atom in or out of the selection, so you can build up a set one atom at a time. Clicking empty space clears the selection.

In **Selection** mode a rubber-band drag selects every atom whose centre falls inside the box; holding Ctrl adds the boxed atoms to the existing selection instead of replacing it. With the viewport focused, Delete or Backspace removes the selected atoms and the bond network is rebuilt immediately.

The status bar reports the selection size, and several tools act on it: the bond-order buttons need exactly two atoms, the bond editor can pull a pair straight from the selection, and **Edit** ⟩ **Selection** can change the element of, or translate, whatever is selected.

4.3 Measuring distances and angles

Measurement modes accumulate clicks on atoms and draw the result directly on the canvas — markers on the chosen atoms, dashed connecting lines, and the value in a floating label. The same value is written to the status bar, for example:

```
Distance Si(0) - Si(4): 2.351 A
Angle O(2) - Si(0) - O(5): 109.47 deg
```

Distance Two atoms; result in Å to three decimals.

Angle Three atoms; the *second* atom clicked is the vertex. Result in degrees to two decimals.

Clicking empty space clears the current measurement, and clicking again after a completed one starts fresh. Dragging still orbits the camera, so you can rotate the structure between clicks to reach an atom hidden behind another. Measurements are cleared automatically when you switch modes or load a different structure.

4.4 Fixed-angle rotations

The toolbar's rotation cluster turns the scene by exact increments about the world axes — useful for reproducible figure orientations.

Angle step An editable spin box, 1–180°, default 15°.

X+ X− Y+ Y− Z+ Z− Counter-clockwise and clockwise rotation about each axis by exactly that step.

Each click animates smoothly over about 200 ms, and rapid clicks compose exactly: four presses of **Z+** at 90° return the scene to where it started. The axes triad follows the rotation, and picking, selection and measurements all stay consistent with what you see.

4.5 Framing, alignment and projection

F Frame the structure: centre it and back the camera off far enough to see all of it. Also on the toolbar and in **View** ⟩ **Alignment**.

Align with XY / XZ / YZ Look straight down the remaining Cartesian axis. Toolbar buttons, or **View** ⟩ **Alignment**. Alignment clears any accumulated fixed-angle rotation, so the result is a true axis-aligned view.

O Toggle orthographic versus perspective projection. The orthographic frustum is matched to the perspective field of view at the camera's target distance, so the structure keeps its apparent size across the switch — and zoom keeps working in both.

> **Tip**
> Orthographic projection is what you want for figures where lattice directions must read as parallel: perspective foreshortening makes a supercell look subtly sheared.

4.6 Inserting atoms and bonds

In **Insertion** mode (I) the toolbar's element button — which shows the active symbol, `C` by default — selects what gets placed. Click it to choose any element from the graphical periodic table.

Click on empty space Inserts one atom of the active element there. The atom lands on the plane through the camera's target point facing the viewer, so what you click is where it appears.

Drag from one atom to another Creates a bond between them, added as a manual bond override.

Click on an existing atom Selects it, as in any other mode.

Both operations are single undo steps.

> **Note**
> Because inserted atoms land on the camera-target plane, the depth of a new atom depends on the current view. Place atoms from an aligned, orthographic view when the exact coordinate matters, or insert roughly and then set exact coordinates with **Edit ⟩ Add Atom**, which takes numeric x, y, z.

4.7 Visual effects

View ⟩ Visual Effects opens a modeless dialog — the scene stays interactive while you tune it — with two depth cues.

4.7.1 Distance fog

Fades distant geometry into the background colour, which makes the depth ordering of a thick slab or a large nanoparticle immediately readable.

Mode Linear (start → end) fades uniformly between two distances; **Exponential (density)** follows $e^{-\rho d}$.

Start distance, End distance The linear ramp, in Å. Defaults 15 and 80.

Density The exponential coefficient, default 0.03.

The fog colour tracks the viewport background automatically, so geometry always fades into whatever backdrop you have chosen.

4.7.2 Depth of field

A post-processing pass that blurs geometry away from the focal plane, in proportion to its circle of confusion.

Blur strength Maximum blur radius in pixels, 1–20.

Focus range The depth band that stays sharp, in Å.

Focus offset Shifts the focal plane relative to the camera target, in Å.

> **Note**
> Depth of field renders the scene into an offscreen buffer before compositing, which means multisample anti-aliasing is unavailable while it is enabled. For final figures, decide which of the two matters more — crisp edges or the depth cue.

CHAPTER 5

Getting Data In and Out

5.1 Supported formats

All structure I/O goes through `ase.io`, so Calango reads and writes every format ASE knows. The file dialog offers the common ones directly:

Family	Extensions
XYZ	`.xyz`, `.extxyz`
Crystallography	`.cif`
VASP	`POSCAR`, `CONTCAR`, `.vasp`
ASE	`.traj`
Quantum ESPRESSO	`.in`, `.pwi`, `.pwo`, `.out`
CASTEP	`.cell`
LAMMPS	`.data`, `.dump`, `.lammpstrj`
Gaussian	`.gjf`, `.com`
SHELX	`.res`

> **Note**
> Extended-XYZ per-atom arrays are imported as *scalar fields* and *vector fields*. A `charges` or `forces` column becomes selectable in the **Custom property** colour mode, and `forces` or `momenta` enable the force and velocity arrow overlays. This is the simplest route for visualising per-atom data computed elsewhere: write it as an extxyz column and open the file.

5.2 Opening structures and trajectories

File ⟩ **Open** ⟩ **Structure** Ctrl+O.

File ⟩ **Open** ⟩ **Trajectory** Ctrl+T.

Both entries read *every* frame in the file. A single-frame file opens as a plain structure tab; a multi-frame file opens as a trajectory with the timeline active. In practice the two entries differ only in the file filter, so either works for either kind of file.

Opening a `.calproj` file — from the command line, your file manager, or the structure dialog — restores the saved session rather than loading a geometry.

5.3 Saving structures and trajectories

File ⟩ Save ⟩ Structure As Ctrl+Shift+S. Writes the currently displayed frame. The name is suggested from the document's own, with the default extension; the format follows the filter you pick, and a name typed without an extension is given the filter's own.

File ⟩ Save ⟩ Trajectory As Ctrl+Shift+T. Writes all frames of the active document as extended XYZ, multi-frame XYZ, ASE `.traj`, or multi-model PDB.

> **Note**
> **Extended XYZ is the default everywhere.** It is pre-selected in every open and save dialog in the application — structures, trajectories, NEB endpoints, dataset imports — and it is what **Save Structure As** writes unless you choose otherwise. The reason is that it is the only format in the list that round-trips everything a Calango document carries: cell and periodicity, per-atom magnetic moments and charges, forces and velocities, and arbitrary extra columns. Plain `.xyz` silently drops the cell; CIF drops the calculator results; POSCAR drops nearly everything but the positions.
> Every other format is still one entry down in the same list, so nothing has become harder to reach — only the *default* changed, to the format that loses nothing.

5.4 The database and preset browser

Build ⟩ From Database opens a two-tab browser.

5.4.1 Presets

Seven benchmark structures ship with Calango, each annotated with a suggested calculator:

Preset	File	Suggested calculator
Diamond (bulk C)	`diamond.vasp`	MACE-MP-0
MoS_2 2H (bulk)	`mos2_2h_bulk.vasp`	MACE-MP-0
Graphene monolayer	`graphene.vasp`	MACE-MP-0
MoS_2 1H (monolayer)	`mos2_1h_monolayer.vasp`	MACE-MP-0
Benzene	`benzene.xyz`	MACE-OFF (or EMT)
Naphthalene	`naphthalene.xyz`	MACE-OFF
Coronene	`coronene.xyz`	MACE-OFF

Select a row and press **Load Selected**, or double-click it.

5.4.2 Materials Project

The second tab fetches structures by Materials Project ID.

API key Masked field, pre-filled from your saved setting or from `MP_API_KEY` in the environment. It is stored as you type.

Material ID For example `mp-149` (silicon). Pressing Enter triggers the fetch.

.env file Shows the current environment file path, with **Browse...** to point elsewhere and **Reload** to re-import the key (Section 1.2.3).

Fetch Structure Downloads the entry and opens it in a new tab, reporting the formula and atom count.

Requires

Needs a network connection and a personal API key from https://materialsproject.org/api.

CHAPTER 6

Building Structures

Everything in the **Build** menu produces a new structure, either replacing the active one or opening in a new tab. All of it runs through ASE, so a working Python environment is a prerequisite throughout this chapter.

6.1 Supercells

Edit ⟩ **Unit Cell** ⟩ **Create Supercell** repeats the active structure along its lattice vectors. Three spin boxes, **Repeat along a/b/c**, each 1–20, default $2 \times 1 \times 1$. The operation is undoable and requires a defined cell along every repeated direction.

6.2 The surface slab wizard

Build ⟩ **Surface Slab** cleaves a slab from the active bulk structure in three guided stages. Unlike a plain Miller-index dialog, each stage shows you the consequence of your choice before you commit.

6.2.1 Stage 1 — surface orientation

Set the Miller indices $(h\,k\,l)$ with three spin boxes (-9 to 9), or work the other way round: drag the red **u** and green **v** handles on the interactive lattice canvas to any two lattice points, and Calango computes the nearest integer $(h\,k\,l)$ for the plane they span. The header shows the current vectors and resulting indices live, and the info line reports $|u|$, $|v|$ and the angle between them.

Rotate the axonometric view by dragging; only non-collinear handle pairs are accepted. **Next** unlocks once a test cut succeeds — (0 0 0) is rejected as not a plane.

6.2.2 Stage 2 — cut, thickness and terminations

Calango builds a reference stack of eight bulk repeats and clusters the atoms into atomic layers (0.10 Å tolerance). The cross-section canvas draws each layer as a dashed line labelled with its z coordinate, with the current selection band highlighted between an orange *top termination* and a teal *bottom termination*.

Click a layer Moves whichever boundary is nearer the click, so you can set either termination directly.

Layers in slab Sets the count numerically.

Thickness The same choice in Å; it snaps to whole atomic layers above the bottom termination.

The info line reads back the selection, for example *layers 1–8 of 16 (8 layers, 12.45 Å)*.

6.2.3 Stage 3 — vacuum and preview

Top vacuum and **Bottom vacuum** (0–80 Å, default 10 each) pad the cell along the surface normal. **Symmetric vacuum (centered slab)**, checked by default, mirrors the top value to the bottom. A live 3D preview shows the finished slab, with atom count, slab thickness and total cell height. **Finish** inserts it as a new tab labelled, for example, *(1 1 1) slab, 8 layers*.

6.3 Liquid and gas interfaces

Build ⟩ Liquid / Gas Interface opens a fluid region on the structure in the current tab and packs it with a liquid, a gas, a mixture, or an ionic solution — the solid/liquid and solid/gas cells that electrochemistry, catalysis and adsorption studies start from. It consumes a structure rather than generating one, so build or open the substrate first; a surface slab from §6.2 is the usual input. The result opens as a new tab and the substrate's own tab is left untouched.

6.3.1 Stage 1 — geometry and region

Interface direction Which lattice vector the region is opened along; *c* for a slab built by this application. The region is bounded by planes normal to the *other* two lattice vectors, so it stays commensurate with the cell even when that cell is tilted.

Region thickness The gap between the top face of the structure and the bottom face of its own periodic image. The cell is grown — or shrunk — to make this *exact*, whatever vacuum the input already carried. Asking for 20 Å of water and silently getting 20 Å plus the slab's existing 15 Å is the usual way to end up with an accidentally dilute interface.

Lateral supercell Repeats the substrate in the two directions that lie *in* the interface plane, before the region is opened. A wider cell is usually what a solvation study needs: the fluid's correlation length is several Å, and a 1×1 surface cell forces it to be periodic on a shorter length than that.

Surface clearance Closest approach between any fluid atom and any substrate atom (default 2.2 Å).

Move the substrate to the bottom of the cell On by default, so the fluid region is one contiguous block. Turn it off when the slab was positioned deliberately — a symmetric slab centred in its cell, for instance.

The summary underneath reports the cell before and after, the fillable volume, and how many water molecules that volume holds at ambient density.

6.3.2 Stage 2 — solvation and mixture

Liquid / gas mixture is a table of species and mole fractions. The fractions are normalised, so {3 water, 1 ammonia} and {0.75, 0.25} request the same mixture. **Amount** chooses between a target mass density and an explicit molecule count.

Ionic species inserts salts or bare ions by *formula unit*. A salt expands into its ions, so three units of $(NH_4)_2SO_4$ is six ammonium and three sulfate and the cell stays neutral by construction.

Bare ions are offered too, for the cases where an unbalanced charge is deliberate; the net charge is reported below the table, in orange when it is not zero.

Ions are placed *before* the solvent and their mass counts against the density target. Both matter: a packing that saturated could otherwise drop an ion and silently change the stoichiometry, and a density target that ignored the ions would give a brine at the density of water with salt added on top of it.

> **Tip**
> A gas at its 1 bar density comes to a fraction of a molecule in a cell of interface size, so picking a gas switches the page to an explicit molecule count. That is the physically meaningful control: in a periodic cell the molecule count *is* the partial pressure.

6.3.3 What the packer does, and what it does not

Molecules are proposed at random positions with uniformly random orientations and kept when they clear everything already placed. Two fluid molecules are excluded on their centres, using a tabulated effective radius per species, with an atom-level floor on top so that two molecules facing each other cannot interlock their hydrogens; the substrate is checked atom by atom, because that surface is fixed and a molecule started inside it cannot relax out. The packing is seeded and therefore reproducible.

> **Requires**
> The result is a legitimate *starting point* — right composition, right density, no overlaps — and it is not an equilibrated liquid. A random packing has no hydrogen-bond network and a potential energy far above the equilibrium one. Quoting a diffusion coefficient, an interface energy or a work of adhesion computed directly on this cell would be reporting a property of the random number generator. Equilibrate with molecular dynamics (§9.1.3) first.

Random sequential packing saturates below the equilibrium density of a hydrogen-bonded liquid; around 90% of the target is typical for water and is perfectly usable. A larger shortfall is reported explicitly after the cell opens, together with what to do about it.

> **Note**
> The species library holds only molecules whose geometry is fixed by symmetry plus a bond length and, where needed, one angle: water, ammonia, HF, H_2S, the common diatomic and small polyatomic gases, the noble gases, the alkali and alkaline-earth cations, the halides, and the molecular ions NH_4^+, H_3O^+, OH^-, NO_3^-, CO_3^{2-}, SO_4^{2-} and PO_4^{3-}. Species that need a real optimised geometry — the alcohols, anything with a torsional degree of freedom — are deliberately absent rather than approximated: a molecule with plausible-looking but wrong internal coordinates produces a cell that nobody notices is wrong until the first relaxation step tears it apart.

6.4 Dislocations

Build ⟩ Dislocation inserts a Volterra line defect into the structure in the current tab by displacing its atoms with a closed-form elastic field. The result opens as a new tab; the parent is left alone.

A dislocation is one operation: cut the crystal along a half-plane bounded by a line, slide the two faces past each other by a lattice vector **b**, and weld them back together. Everything the

module does is that operation, evaluated analytically at every atom.

6.4.1 Stage 1 — type and geometry

Line direction names the Cartesian axis the dislocation line runs along; the other two axes follow in cyclic order as $(\mathbf{e}_1, \mathbf{e}_2)$. For a line along z those are (x, y), and an edge dislocation then has $\mathbf{b}$ along x with its glide plane normal along y — so naming the line names everything.

Burgers vector is $|\mathbf{b}|$ in Å. Use a lattice repeat of the host: a perfect dislocation's Burgers vector *is* a lattice vector, and any other length leaves a stacking fault trailing behind the line that this module does not model. **Line position** places the line in the plane normal to it, as a fraction of the structure's extent. The default is deliberately a little off centre — linear elasticity is singular *on* the line, so a line placed exactly on a lattice site evaluates that atom's displacement at infinity.

Edge Single edge dislocation. $\mathbf{b} \perp$ line, the crystal carries an extra half-plane ending on it, and the atom count is unchanged — this is the elastic field, not a plane of atoms inserted by hand.

Screw Single screw dislocation. $\mathbf{b} \parallel$ line; the lattice planes around it form one continuous helical ramp, and only the component along the line moves.

Glide Two edge dislocations of opposite sign in the *same* glide plane — what a dislocation leaves behind after gliding a distance d. Between the cores the crystal above the glide plane has slipped by exactly one $\mathbf{b}$ relative to the crystal below. Conservative: no atom is created or destroyed.

Climb Two edge dislocations of opposite sign stacked normal to the glide plane, with a platelet of *missing* material between them — a collapsed vacancy disc one $\mathbf{b}$ thick. Climb is mass transport, so this is the one construction that changes the atom count, and that change is how you tell it happened.

Anisotropic A single dislocation solved from the full elastic tensor via Stroh's sextic formalism. The only option that can describe a *mixed* dislocation — one whose Burgers vector has both an edge and a screw component — and the only one that is correct for a strongly anisotropic crystal.

6.4.2 Stage 2 — elasticity

The isotropic types read a single **Poisson ratio**; a screw dislocation does not depend on it at all. The two dipoles read a **Core separation** — the distance the dislocation travelled, for glide, and the height of the vacancy platelet, for climb.

The anisotropic type reads an elastic tensor instead, built from the constants of a chosen **Elastic symmetry** (cubic, hexagonal or isotropic), with room-temperature single-crystal presets for Cu, Al, α-Fe, Ni, W and Si. Only the *ratios* of the constants reach the displacement field — the absolute scale cancels — but keeping physical units means numbers can be read straight off a table. The **Burgers direction** gives $\mathbf{b}$'s components in the dislocation frame: $(1, 0, 0)$ is a pure edge, $(0, 0, 1)$ a pure screw, anything between a mixed dislocation.

The page runs the real builder on every change and reports what it produced — atom count, largest displacement, closest resulting pair, and any warnings. It is not a model of what **Finish** will do; it is what **Finish** does.

6.4.3 Periodicity

A single dislocation has a net Burgers vector, so its displacement field is multivalued: it cannot be made periodic in the two directions normal to the line, whatever the cell. Edge, Screw and

Anisotropic therefore leave the cell periodic *only along the line* and say so — which is the correct setup for a cylinder with free lateral surfaces, the usual way core structures are computed.

A dipole has zero net Burgers vector, so Glide and Climb stay periodic in all three directions. Reaching that takes one extra step: superposing two opposite fields cancels the net Burgers vector but leaves the average plastic distortion the pair carries, so a compensating uniform distortion — a simple shear for glide, a uniaxial contraction for climb — is applied to the atoms *and* to the cell vectors. Without it the construction looks perfectly normal in a viewer and carries a $|\mathbf{b}|$-sized discontinuity at the periodic boundary.

Requires

Nothing here is relaxed, and linear elasticity is singular at the line. The handful of atoms nearest the core arrive at positions that are qualitatively right and numerically meaningless. Relax the core before quoting anything about it — a core energy, a Peierls barrier, a dissociation width.

Note

Only the vacancy sense of climb is built (material removed). The interstitial sense would mean inserting a partial plane of atoms, which needs the periodicity of the host lattice and not merely its Burgers vector; it is refused rather than approximated. The module also reports when the requested $|\mathbf{b}|$ is smaller than the plane spacing it was meant to remove, in which case the result is an elastic dipole with no mass transport at all.

6.5 Solid interfaces

Build ⟩ Solid Interface builds the constructions that put two pieces of crystal next to each other. All five are the same operation seen from different distances: decide, for every point of space, which crystal orientation occupies it; fill each region with that lattice; reconcile the seams.

Stacking fault Everything above the chosen plane is shifted rigidly by a fraction of an in-plane lattice vector. The cheapest planar defect there is, and the one whose energy decides how readily a dislocation splits into partials.

Twin boundary The half above the plane is discarded and replaced by the mirror image of the half below it. A coherent twin: every atom on the boundary is shared between the two orientations, so the interface carries no free volume. Anything that existed only in the replaced half is gone — that is what twinning does.

Bicrystal Two grains with independent orientations filling the two halves of a box built from the parent cell. The general grain boundary.

Polycrystal A Voronoi tessellation: N seeds scattered in the box, each with a uniformly random orientation, every point of space joining the grain whose seed is nearest.

Multi-phase polycrystal The same tessellation, with each grain drawing its lattice from a different phase. Every grain is single-phase; the interfaces between them are phase boundaries.

6.5.1 Stage 1 — interface and plane

Boundary normal names a *lattice* vector rather than a Cartesian axis, which is what keeps the boundary plane — spanned by the other two lattice vectors — commensurate with the cell by

construction. **Boundary position** places it as a cell fraction.

The stacking fault reads a **Fault vector**, given as fractions of the two in-plane lattice vectors; $\frac{1}{3}$ along one of them is the classic partial-dislocation fault vector of a close-packed plane. Both planar defects can open a **Boundary gap** perpendicular to the interface, which gives a relaxation somewhere to start from instead of two planes on top of each other.

The space-filling kinds read a **Merge tolerance** instead: atoms landing closer than this to one already placed are deleted. Two crystals meeting at an arbitrary angle always overlap somewhere, and without this the boundary is a pile-up rather than a boundary. Too large and it starts eating the grains themselves — watch the reported atom count.

6.5.2 Stage 2 — grains

Box repeats sets the size of the constructed cell in multiples of the parent. A bicrystal reads two rotations about the boundary normal, so the pair forms a twist boundary and the misorientation is their difference; 36.87° is the Σ5 coincidence-site relationship of a cubic lattice. A polycrystal reads a **Grains** count and a **Random seed**; the same seed rebuilds the same cell exactly, which is what makes a published polycrystal reproducible. A multi-phase polycrystal adds a table of the open workspace tabs with a weight each — a weight of zero excludes that phase entirely.

Every atom of a space-filling construction carries its grain index and phase index as per-atom scalar fields, so the tessellation can be *seen* in the viewport (Representation ⟩ colour by scalar field) rather than assumed.

Requires

A periodic cell cannot contain an odd number of parallel interfaces. Insert one boundary at the middle of a cell and you have inserted two: the one you asked for, and the one where the top face meets the bottom face of the next image. That is not a defect of the implementation, it is what periodic boundary conditions mean, and every result reports both. A calculation that attributes the whole excess energy of the cell to "the" boundary is out by a factor of two.

Note

Rotating a lattice and demanding it still fit the box is a strong condition: only the coincidence-site (CSL) misorientations satisfy it exactly. At any other angle the two crystals meet the periodic boundary out of register. The mismatch is measured and reported in Å rather than hidden, so a bicrystal built at an arbitrary angle announces what it is.
Twinning additionally requires that the stacking vector be perpendicular to the boundary plane. Reflecting a tilted cell would produce a half-crystal that is periodic with a *different* cell, so the construction is refused with a message asking for an orthogonalised cell rather than returning a box that describes neither half.

Requires

Nothing here is relaxed. Grain boundaries have structure — free volume, reconstruction, segregation — that no geometric construction produces. These are starting points.

6.6 Nanomaterials

Build ⟩ Nanomaterial Builder generates four families of low-dimensional carbon and TMD structures. Pick the type at the top; the form below changes to match.

Graphene sheet Lattice constant a (default 2.46 Å), repeats $n_x \times n_y$ (default 4×4), vacuum (default 10 Å).

Graphene nanoribbon Width and length in unit cells (default 6 each), **Edge type Zigzag** or **Armchair**, and **Hydrogen-terminate edges** (on by default).

Carbon nanotube Chiral indices n and m (default 6, 6), length in unit cells, C–C bond length (default 1.42 Å), vacuum (default 8 Å). $(0, 0)$ is rejected.

TMD monolayer (MX_2) An editable formula combo — `MoS2`, `MoSe2`, `WS2`, `WSe2`, `MoTe2`, `NbSe2`, or anything else you type — **Phase** `2H` or `1T`, lattice constant (default 3.16 Å), X–X thickness (default 3.19 Å), repeats and vacuum.

6.7 Metallic nanoparticles

Build ⟩ Metallic Nanoparticle (Wulff) builds clusters two ways. Choose the element from the periodic table, then a mode.

6.7.1 Wulff construction

The equilibrium shape of a crystal is the one minimising total surface energy at fixed volume; the Wulff construction produces it from the relative surface energies of the exposed facets.

Facet table One row per facet: h, k, l and γ (relative surface energy). Defaults are $(111) = 1.0$, $(100) = 1.1$, $(110) = 1.2$. **Add Facet** and **Remove Selected** edit the list.

Target atoms The requested size, default 200. The construction lands on the nearest achievable closed shape.

Size rounding `closest`, `above` or `below`.

Lattice `fcc`, `bcc` or `sc`.

Only the *ratios* of the γ values matter. Lowering $\gamma(111)$ relative to $\gamma(100)$ grows the (111) facets and drives an fcc metal from a cube towards a truncated octahedron.

6.7.2 Spherical cluster

Carves every atom within a cutoff radius of the centre out of a bulk lattice — `fcc`, `bcc`, `sc` or `hcp`. Set **Radius** (default 10 Å). Useful when you want a specific diameter rather than a thermodynamic shape.

Both modes centre the cluster with 6 Å of vacuum and clear periodicity. **Lattice constant** may be left at **ASE default** to use the tabulated value for the element.

6.8 Special quasirandom structures

Build ⟩ Special Quasirandom Structure (SQS) builds a substitutional alloy whose short-range order approximates a truly random solid solution — the standard way to model a disordered alloy in a small periodic cell.

Supercell Repetitions $n_1 \times n_2 \times n_3$, default $2 \times 2 \times 2$.

Replace element Whose sites form the alloy sublattice.

Composition Target occupancies as symbol:fraction pairs, e.g. `Cu:0.75, Au:0.25`. Fractions are normalised and rounded to whole atoms by largest remainder.

First shell cutoff, Second shell cutoff The pair shells whose Warren-Cowley parameters are driven toward zero. Defaults 3.2 and 4.8 Å; set the second to **off** to use one shell.

MC steps, Random seed Anneal length and reproducibility.

If `icet` is importable, Calango uses its `generate_sqs`; otherwise it falls back to a built-in simulated annealer that minimises $\sum \alpha^2$ over the chosen shells using only NumPy and ASE neighbour lists. The resulting tab is labelled with the backend that produced it, and the internal backend also reports its residual objective — a value near zero means the decoration is essentially ideal.

> **Tip**
> Verify the result with **Analysis ⟩ Warren-Cowley analysis** (Section 14.6): a good SQS shows $\alpha \approx 0$ for the shells you optimised.

6.9 Normal modes and phonons

Build ⟩ Normal Modes / Phonon Builder sets up a finite-displacement vibrational calculation. Calango detects whether the structure is periodic and adapts:

Periodic Supercell finite displacements via `ase.phonons`, giving dispersion and DOS.

Isolated molecule Γ-point normal modes via `ase.vibrations`, giving frequencies and per-mode animation trajectories that Calango opens directly.

6.9.1 Parameters

Mode **Run vibrational calculation**, or **Generate displaced structures only (new tab)** to export the displacement set for an external DFT code.

Supercell (a × b × c) Default $2 \times 2 \times 2$, periodic structures only.

Displacement δ Default 0.01 Å.

Displaced structures A read-only count: $6N + 1$ for N supercell atoms.

Calculator EMT, Lennard-Jones or MACE — deliberately limited to calculators needing no external binary or pseudopotentials.

Band-path points, DOS k-grid Sampling for the dispersion and the phonon density of states (defaults 100 and 20).

Like the calculator dialog, the generated ASE script is shown in an editable pane and can be saved with **Save Script...**.

Outputs are Γ-point frequencies in the job console, plus `phonon_bands.csv` and `phonon_dos.csv` in the job directory.

> **Note**
> Displacements are applied to every atom of the expanded supercell ($6N_{\text{supercell}} + 1$ configurations), following the standard finite-displacement recipe without symmetry reduction. For large supercells this is the dominant cost; symmetry-reduced displacement sets are a planned improvement.

6.10 Structure perturbation and noise

Build ⟩ Structure Perturbation / Noise adds random displacements — for shaking a structure out of a symmetric saddle point, or for generating training ensembles for machine-learning potentials.

Distribution **Gaussian (normal)** or **Uniform**.

Amplitude Default 0.05 Å. For Gaussian this is σ per component; for uniform it is the half-width.

Random seed Default 42, for reproducibility.

Perturb atomic positions On by default.

Perturb unit cell vectors (random strain) Atoms follow the cell affinely, preserving fractional coordinates.

Frames 1 perturbs in place; more generates a trajectory.

Accumulation **Independent** draws each frame from the original structure; **Cumulative** performs a random walk from the previous frame.

A multi-frame run opens as a trajectory whose frame 0 is the unperturbed original, is registered in the process manager, and is checkpointed to `perturbed.extxyz` in the managed temporary directory (Section 3.4.1) — so the ensemble can be reloaded later and fed to the analysis or dataset tools.

CHAPTER 7

Editing Structures

Every operation in this chapter is a single undo step. **Edit** ⟩ **Undo** (Ctrl+Z) and **Edit** ⟩ **Redo** (Ctrl+Shift+Z) walk a per-document history up to 50 states deep.

7.1 Atoms

Edit ⟩ **Add Atom** Ctrl+Shift+A. Choose the element from the graphical periodic table and type exact x, y, z coordinates. For placing atoms by eye instead, use Insertion mode in the viewport (I).

Edit ⟩ **Selection** ⟩ **Change Element of Selection** Replaces the species of every selected atom, again via the periodic table.

Edit ⟩ **Selection** ⟩ **Translate Selection** Shifts the selection by a Δx, Δy, Δz vector in Å.

Edit ⟩ **Delete Selected Atoms** Delete. Removes the selection; with the viewport focused in Selection mode, Backspace does the same.

Deleting atoms re-indexes everything that referenced them — bond overrides, bond orders and per-atom fields all follow correctly.

7.2 The Edit Structure dialog

Edit Structure... in the **Structure** panel opens a two-section editor over a working copy: the unit cell above, and **Atomic Configurations** below.

7.2.1 Coordinates

The table shows *one* coordinate triple, and the **Fractional coordinates** checkbox decides which. Unchecked — the default — gives Cartesian x, y, z in Å; checked converts the values in place and relabels the columns u, v, w. Both are editable: a fractional edit recombines all three components of that row and converts back, so changing u alone does not discard v and w.

The checkbox is disabled without a unit cell. Fractional coordinates there are not merely unavailable, they are undefined.

> **Note**
> One triple rather than two side by side is what makes room for the extended per-atom properties below. It also removes a real hazard of the old layout: with $x \dots w$ all editable

at once it was possible to type into a Cartesian cell and a fractional cell of the same row before either was committed, and only one of the two edits survived.

7.2.2 Extended per-atom properties

Any per-atom array the structure carries gets its own column, to the right of the magnetic moments: partial charges (`initial_charges`), velocities, forces — anything an extended-XYZ file put in `atoms.arrays`. A scalar array gives one column; a vector array gives three, suffixed x, y, z. Columns appear and disappear with the data, so a plain `.xyz` opens with none and an MD frame opens with several.

Most of these columns are **read-only**, and deliberately: they are results. A charge came out of a Bader partitioning; typing over it in a geometry editor would leave a frame whose arrays no longer correspond to anything that was computed, and the file would carry no sign of it.

Velocities and forces are the two exceptions, editable like the coordinates, because both are legitimately an *input* as well as a result: hand-set velocities are the initial conditions of a molecular-dynamics run, and hand-set forces are a way to assemble force-field training frames or to exercise a force-arrow viewer. Edits write into the structure's own arrays, so a save to extended XYZ carries them (velocities through ASE's momenta), and a derived |forces| magnitude column follows the edit rather than going stale. The magnetic moments likewise remain editable, through the spin-mode columns.

7.2.3 Sorting

Sort by reorders the atoms by element or by any coordinate, ascending or **descending**.

This *renumbers* the atoms — it permutes the structure itself, not just the view. That is normally the point: grouping by element is what VASP's POSCAR/POTCAR pairing wants, and sorting by z gives a slab a layer-ordered index list you can select ranges from. Everything index-aligned travels with its atom: per-atom scalar and vector fields, magnetic moments, residue annotation, bond overrides and manual bond orders.

The sort is stable, so atoms that tie on the key — every atom of one element, every site in one layer — keep their existing relative order rather than being reshuffled on each press.

7.2.4 Spin polarization

The **Spin polarization** dropdown adds editable magnetic-moment columns to the table:

Unpolarized No columns. This means *no moments*, not moments of zero — an all-zero column would still switch a calculator into spin-polarized mode and cost the run its speed for nothing.

Collinear Spin-Polarized One signed **m (μ_B)** column. The sign is what makes a seed antiferromagnetic rather than ferromagnetic, which is why it is set per atom rather than as one number.

Non-collinear Spin Three columns, **mx**, **my**, **mz**.

What you type is stored as the structure's *initial* magnetic moments. It travels with the geometry into every calculation — the wizards no longer ask for a list of moments, they read these — and it can be drawn in the viewport through **Spatial References** ⟩ **Vectors** ⟩ **Vector overlay** → **Initial magnetic moments**.

> **Note**
> That overlay is deliberately separate from **Magnetic moment**, which shows the moments a calculation *produced*. A guess and a result are different claims, and drawing one as the other is how a seeded antiferromagnet gets reported as a converged one. The two even use different arrow colours.

The dialog opens on whatever mode the structure is already in, so a magnetic structure shows its moments instead of hiding them behind a dropdown you would have to know to change. The summary line reports the net moment alongside $\Sigma|m|$: a column of 2.2 and a column of ± 2.2 look nearly identical and mean opposite things.

7.3 The periodic table selector

Wherever an element must be chosen, Calango opens a graphical periodic table: $Z = 1$ to 118 in the standard 18-group layout, f-block below, coloured by chemical family. One click selects and closes.

7.4 Bond perception

By default Calango perceives bonds by distance: atoms i and j are bonded when

$$d_{ij} < t \cdot \left(r_{\mathrm{cov}}(i) + r_{\mathrm{cov}}(j)\right),$$

with covalent radii from Cordero *et al.* and a tolerance t you control. With a periodic cell the minimum-image convention applies, so bonds across cell boundaries are found and drawn correctly.

Edit ⟩ **Bond Editor** (Ctrl+B) exposes both the automatic rule and manual overrides:

Automatic bond detection Turn it off to draw *only* manually added bonds — the right choice for coarse-grained or schematic figures.

Covalent cutoff multiplier The tolerance t, 0.50–2.50, default 1.15. Applied live, so you can dial it until the picture is right.

Manual Bonds Two 1-based atom-index spin boxes, or **From Selection** to fill them from a two-atom viewport selection. The info line shows the current pair's distance next to the automatic cutoff for that pair, which makes it obvious whether a bond is borderline.

Add Bond Forces a bond regardless of distance.

Suppress Bond Removes an automatically perceived bond.

Active overrides Lists every override as *added* or *suppressed*, with the pair and its distance. **Clear Override** and **Clear All** undo them.

Overrides live on the structure and are saved in the project file.

7.4.1 Rules apply to the whole trajectory

Every rule entered in the Bond Editor is applied to *every frame* of the active trajectory, not only to the frame on screen. Bonding is a statement about the chemistry of a system, and a system

does not change its chemistry between two samples of the same run — an override that existed on frame 0 and vanished on frame 1 was never a physical statement, only an artifact of which frame happened to be displayed when the button was pressed. It also made every trajectory-wide export (the extended-XYZ writer, the ray-traced film, the animated GIF) disagree with the viewport.

The two kinds of rule propagate *differently*, and the difference is the point:

By Atomic Indices Names atoms. An atom keeps its index for the whole run, so the same pair — and the same bond order — is marked on every frame.

By Chemical Elements Names a *geometric condition*: every Si–O pair between 1.4 and 1.9 Å. The rule is therefore re-evaluated against each frame's own coordinates. Copying the first frame's match list forward would freeze a bond onto a pair that has since dissociated — precisely the event a reactive trajectory is being watched for, rendered invisible.

The scope is stated on the panel where the rules are entered, and the element-rule match count says which frame it was counted on. Frames whose atom count differs from the displayed one are skipped and reported: an index there refers to a different atom.

> **Note**
> Hydrogen bonds are the exception, and deliberately so. They are perceived geometrically rather than stored as overrides, so they re-evaluate whenever the geometry moves — they already follow a trajectory instead of freezing at the frame they were detected on.

7.5 Bond orders

Calango never guesses multiple bonds. Every perceived bond starts as a single bond, and orders are assigned deliberately — distance-based perception of double and triple bonds is unreliable, particularly for inorganic and metallic systems where short contacts are common.

To assign one, select exactly two atoms in the viewport (Ctrl+click the second) and use the **Bond order** buttons in the **Representation** panel: **Single**, **Double**, **Triple**. The buttons are disabled until a pair is selected.

An order of two or three renders as that many parallel cylinders, and also forces the bond to exist even if the pair lies outside the automatic cutoff. Orders persist in project files.

> **Note**
> A consequence worth knowing: molecules like benzene or CO_2 render with all-single bonds when first opened. Assign the orders you want to display; they are a presentation property, not an input to any calculation.

CHAPTER 8

Appearance and Representation

8.1 The Representation panel

8.1.1 Representation mode

Mode selects how atoms and bonds are drawn:

Ball-and-Stick Spheres at a fraction of the covalent radius, bonds as cylinders. The default.

Space-filling (CPK) Spheres at approximately the van der Waals radius; bonds are hidden inside the atoms.

Wireframe Bonds only, drawn as lines with points at the atom positions — the cheapest mode for very large systems.

Atom radius and **Bond width** scale everything globally, 0.20–3.00, each with a slider and a synchronised spin box for exact values.

8.1.2 Colouring atoms

Color by offers four mappings, applied consistently to atoms and to their halves of each bond:

Element (CPK) The Jmol palette, with per-element overrides.

Coordination number (CN) Discrete coordination numbers on a gradient.

Generalized CN (GCN) Continuous generalised coordination numbers — excellent for distinguishing terraces, steps, edges and vertices on nanoparticles and slabs.

Custom property Any per-atom scalar field the structure carries: charges, force magnitudes, extended-XYZ columns, or a computed field such as `local_entropy`. **Property** selects which.

For the three scalar modes, **Gradient** chooses the colour map — **Viridis**, **Plasma**, **Turbo**, **Inferno**, **Magma**, **Cividis**, **Hot**, **Afmhot**, **Coolwarm**, **Rainbow** — and **Invert palette** reverses the scalar-to-colour mapping, matching matplotlib's `_r` variants. **Range** reads back the min and max of the active field, so the legend is never ambiguous.

> **Tip**
> Viridis and Cividis are perceptually uniform and colour-vision safe; Coolwarm is the right choice for signed quantities such as charges or potentials, where the zero point should read as neutral. Rainbow is included because reviewers sometimes ask for it, not because it is a good default.

Colour mapping is recomputed for every frame during trajectory playback, so CN, GCN and property maps stay live throughout an animation.

8.1.3 Bonds and vectors

Gradient bond coloring Blends each bond smoothly from one atom's colour to the other's, instead of the classic half-and-half split.

Force vectors, Velocity vectors Draw per-atom vectors as lit 3D arrows. Each is enabled only when the structure actually carries that field; the tooltip explains what is missing when it does not.

Vector scale Arrow length in Å per field unit, 0.05–200, default 10.

8.1.4 Per-element styling

Element Settings... opens a table of the elements present in the structure. Each row offers a colour swatch and a radius scale (10–300 %, where exactly 100 % means "no override"), plus a **Reset** button. **Reset All** clears every override at once.

Background sets the viewport background colour. Distance fog, if enabled, automatically fades into whatever you choose.

8.2 Unit cell and axes

The **Unit Cell & Axes** panel (zone 12):

Show unit cell Also on **View** ⟩ **Overlays**.

Cell color Wireframe colour.

Cell line width 1.0–8.0, default 2.0. A width of 1 draws plain GL lines; anything larger renders the edges as lit tubes.

Show axes triad The corner orientation indicator.

Axes style **Cartesian (X, Y, Z)** or **Lattice vectors (a1, a2, a3)**.

Axes size On-screen size of the triad, 48–240 px.

> **Note**
> Core-profile OpenGL clamps hardware line width to one pixel, which is why thick cell edges are drawn as tube geometry rather than wide lines. The tubes are lit like the rest of the scene, so they also read correctly in ray-traced exports.

8.3 Lighting

The **Lighting** panel (zone 9) edits up to four directional lights with Blinn-Phong shading. The default studio setup is three lights: a warm key carrying the speculars, a cooler fill from the opposite side, and a back (rim) light that separates atom silhouettes from the background.

Select a light in the list, then edit:

Direction (view space) Three components, −5 to 5. Because directions are in view space, the lighting stays fixed relative to the viewer as you orbit — the structure turns, the studio does not.

Ambient, Diffuse, Specular Per-light colours.

Add appends a fill light from the opposite side; **Remove** deletes the selected one. At least one light always remains.

CHAPTER 9

Running Simulations

9.1 The calculation dialog

Simulation ⟩ **New Calculation (ASE)** (Ctrl+R) is the main entry point. The left half is a form; the right half is the ASE script that form produces, live and editable.

9.1.1 Calculators

Calculator	Notes
EMT	Effective medium theory. Fast, ships with ASE, useful for testing a workflow before committing compute.
Lennard-Jones	Ships with ASE.
Quantum ESPRESSO	DFT; needs `pw.x` and pseudopotentials.
VASP	DFT; needs a license and POTCARs.
MACE	Machine-learning potential; needs `mace-torch`.
GPAW	DFT as a Python package.
SIESTA	DFT; needs the `siesta` binary and pseudopotentials.
ORCA	Quantum chemistry; needs the ORCA binaries.

The DFT entries generate working script skeletons with clearly marked `EDIT ME` lines where you must supply pseudopotential paths or launch commands.

9.1.2 Tasks

Single-point energy One energy and force evaluation.

Geometry optimization (BFGS) Controlled by **Force convergence (fmax)** (default 0.05 eV/Å) and **Max optimization steps** (default 200). Writes `opt.traj`.

Molecular dynamics See below. Writes `md.traj`.

9.1.3 Molecular dynamics ensembles

Ensemble	Thermostat / barostat	Key parameters
NVE	Velocity Verlet	timestep
NVT	Langevin (default)	temperature, friction
NVT	Andersen	temperature, collision probability
NVT	Berendsen	temperature, τ_T
NVT	Nosé–Hoover chain	temperature, τ_T
NPT	Berendsen	temperature, pressure, τ_T, τ_P
NPT	Nosé–Hoover / Parrinello–Rahman	temperature, pressure, τ_T, τ_P

Shared parameters: **Temperature** (default 300 K), **Timestep** (default 1 fs), **MD steps** (default 1000), **Thermostat coupling** (default 100 fs), **Barostat coupling** (default 1000 fs) and **Pressure**

(default 0 GPa). Fields enable themselves only for the ensembles that use them.

> **Note**
> Every MD run initialises velocities from a Maxwell-Boltzmann distribution and then removes the net centre-of-mass momentum, so the system does not drift as a whole. For isolated (non-periodic) systems the net angular momentum is removed as well. Without this, a slow overall translation contaminates every subsequent analysis — diffusion coefficients most of all.

9.1.4 Simulated annealing

The **Mode** selector at the top of the dynamics page switches the run between **Constant temperature** — ordinary MD at a fixed setpoint — and **Annealing**, which sweeps the thermostat target from an **Initial temperature** to a **Final temperature** over the course of the run. Everything else is unchanged: the same integrator, the same constraints, the same sampling, the same streamed trajectory. Only the target moves, and it is retargeted on *every* step rather than once per sampling interval.

Three schedules are offered, written in terms of the run fraction $x = n/N$ where n is the current step and N the total:

Schedule	$T(x)$
Linear	$T_0 + (T_1 - T_0)\,x$
Exponential	$T_1 + (T_0 - T_1)\dfrac{e^{-kx} - e^{-k}}{1 - e^{-k}}$
Logarithmic	$T_0 + (T_1 - T_0)\dfrac{\ln(1 + kx)}{\ln(1 + k)}$

All three are *endpoint-exact*: $T(0) = T_0$ and $T(1) = T_1$ whatever the coefficient k. That is deliberate — a schedule that only approaches its target asymptotically ends the run at a temperature nobody asked for, and the discrepancy is invisible in a plot that has been cooling for 50 ps. The coefficient k (**Ramp curvature**) is a curvature rather than a rate constant with units: it says how far the ramp bends away from a straight line, and every schedule degenerates to Linear as $k \to 0$. Linear has no free coefficient, so the field is disabled there.

Exponential moves most of the way early and crawls at the end, which is the shape wanted for quenching a melt. Logarithmic is the slowest of the three near the target; it is the practical, endpoint-exact relative of the Geman–Geman $\sim 1/\ln n$ schedule, the one with the global-optimum convergence proof and a run length nobody can afford.

The page restates the chosen schedule as the temperatures it actually produces, at 0/25/50/75/100 % of the run, together with the elapsed time the ramp spans. A named curve is not a number anyone can check; "1000 → 631 → 419 → 315 → 300 K over 20 ps" is.

> **Note**
> Annealing needs a thermostat to retarget, so NVE is withdrawn from the ensemble list while the mode is selected (and the ensemble is moved to Langevin if NVE was the one chosen). The initial Maxwell-Boltzmann velocities are drawn at the *initial* temperature, not at the constant-temperature field, so a 1000 K quench starts where the schedule says it starts instead of spending its first picosecond catching up.

Tip

The MD Viewer plots the moving setpoint underneath the measured temperature, and the CSV export carries it as its own column (`target_temperature_K`). That second trace is what answers the only question an annealing run is asking — is the system following the schedule? — and it is invisible from the temperature alone. A constant-temperature run has no such series; it reports its single setpoint as a dashed reference line instead.

9.1.5 DFT and ORCA parameters

GPAW and VASP share the **Plane-wave cutoff** row and the **k-point grid**; Quantum ESPRESSO and SIESTA each have a settings group of their own (§9.1.6, §9.1.7), because the parameters that decide their basis are not the same parameters.

The cutoff is a property of the *plane-wave* basis, so under GPAW it is shown only in **PW** mode. In **FD** the basis is the real-space grid (controlled by **Grid spacing h**) and in **LCAO** it is the atomic orbital set — a cutoff there converges nothing, which is a trap worth not laying.

9.1.6 Quantum ESPRESSO settings

QE is a plane-wave code with a *dual* grid, and that is what its group is built around.

ecutwfc The cutoff for the wavefunctions, in Rydberg — the unit `pw.x` reads, so nothing is converted behind your back. Converge it against energy *differences*, not the absolute energy, which keeps falling long after everything you care about has stopped moving.

ecutrho The cutoff for the charge density, on QE's second grid. `auto` means QE's own default of 4× ecutwfc.

XC functional `input_dft`.

Occupations `smearing`, `fixed`, or one of the two tetrahedron methods. The smearing function and width appear only for `smearing`: the others take no width, and writing `degauss` beside them is how a QE input gets silently ignored.

Smearing Marzari–Vanderbilt ("cold") is QE's recommended default. Fermi–Dirac is the odd one out — it is a physical electronic temperature, not a convergence aid.

conv_thr The SCF threshold in Ry. A phonon or Born-charge run wants 10^{-10} or tighter, because it differentiates this quantity.

Requires

The 4× default for **ecutrho** is correct for *norm-conserving* pseudopotentials and badly under-converged for ultrasoft or PAW, whose augmentation charges are much harder than the wavefunctions and want 8–12×. Leaving it on `auto` with a USPP library is the single most common way a QE run comes out quietly wrong. The note under the row restates the effective dual so the number is always in front of you.

9.1.7 SIESTA settings

SIESTA has no plane-wave cutoff, and the option is gone. Its basis is a finite set of numerical atomic orbitals; there is no plane-wave expansion to truncate. What the removed field used to do was set `MeshCutoff` — so raising it to "converge the basis" refined a real-space grid while the basis stayed exactly as small.

XC functional Resolved by ASE into SIESTA's `XC.functional` / `XC.authors` pair. `PZ`, `CA` and `PW92` are the LDA parametrizations; `DRSLL` and `VV` are van der Waals functionals.

Basis type `PAO.BasisType` — *how* the multiple-ζ orbitals are generated. `split` is the standard scheme and what almost every published SIESTA calculation uses.

Basis size `SZ`, `SZP`, `DZ`, `DZP`, `TZP` — how many orbitals per valence shell, and whether polarization orbitals are added. `DZP` is the standard production basis. **This is the parameter that plays the role a plane-wave cutoff plays elsewhere: it is what you converge.**

Energy shift `PAO.EnergyShift`: the energy by which confinement raises each orbital, which is what fixes its cutoff radius. *Smaller* means longer-ranged orbitals, a better basis and a more expensive run — the second knob to turn after the basis size. SIESTA's default is 0.02 Ry ≈ 0.27 eV.

Mesh cutoff `MeshCutoff`: the fineness of the real-space grid the Hartree and exchange-correlation terms are integrated on. Not a basis-set parameter. An under-converged mesh shows up as an "egg-box" force error as atoms move across grid points.

Under **Spin Configurations**, only the polarization *mode* is chosen here. The per-atom initial moments are a property of the structure: set them in **Edit Structure…** (§7.2), where you can see which atom is which, and they travel with the staged geometry into every calculation.

ORCA has its own group:

ORCA method Editable: `B3LYP`, `PBE0`, `r2SCAN`, `wB97X-D3`, `HF`, `MP2`, or any ORCA keyword you type.

ORCA basis Editable: `def2-SVP`, `def2-TZVP`, `def2-TZVPP`, `cc-pVDZ`, `cc-pVTZ`, …

Charge −10 to 10.

Multiplicity $2S + 1$, 1 to 11.

Solvation None (gas phase), **CPCM** or **SMD**, with **Solvent** naming the medium (water, acetonitrile, toluene, …).

ASE writes the ORCA `.inp` file into the job directory and parses the output. The generated script contains an `EDIT ME` line for the path to your `orca` binary.

9.1.8 VASP parameters

Selecting VASP opens a **VASP settings** group exposing the primary INCAR tags. It is built into the shared calculator page, so every wizard that offers VASP shows the same controls.

POTCAR directory The PAW datasets, i.e. `VASP_PP_PATH`. Remembered across sessions and shared by every VASP run, since it describes the installation rather than a job. Both layouts work: the canonical one with a `potpaw_PBE/` level inside, and the flat one where the element folders sit directly in the directory — for the latter the generated script builds a symlink shim, because ASE cannot be told to look anywhere else.

XC functional ASE's `xc`, which expands to the matching `GGA`/`METAGGA` tag and its recommended defaults.

PREC / ALGO Grid accuracy and the electronic minimization. `Accurate` is the default: `Normal`'s coarser grid puts egg-box error into the forces, which is what a relaxation is most

sensitive to.

NELM / EDIFF The SCF iteration cap and its energy threshold.

LREAL `Auto` is a large speed-up above roughly 20 atoms; `False` is exact and right for small cells.

Relaxation driver Who takes the ionic steps. Exactly one side may: VASP relaxes internally when `IBRION`/`NSW` say so, and ASE's optimizers relax anything that returns forces, so with both enabled *every force evaluation ASE asked for ran a complete VASP relaxation.*

ASE optimizer (the default) pins VASP to `IBRION` $= -1$, `NSW` $= 0$ and lets ASE take the steps. This is the mode the rest of the application is built around — geometry constraints, the variable-cell filters, the live streamed trajectory and the per-step metrics all come from ASE driving.

VASP internal relaxation creates no ASE optimizer at all; the tags below *are* the relaxation. Much faster per step, because VASP keeps the wavefunction and charge density between ionic steps instead of restarting — but the steps happen inside a single call, so they cannot be streamed or constrained from here. The ionic path is recovered from `OUTCAR` afterwards, so the trajectory tab and the Geometry Optimization Viewer work either way.

IBRION / ISIF / EDIFFG Shown only for a VASP-driven relaxation, since they describe nothing otherwise. A negative `EDIFFG` is a force criterion in eV/Å; a variable-cell relaxation is raised to `ISIF` ≥ 3 automatically.

Write `LCHARG` (on — CHGCAR is what every density analysis reads), `LWAVE`, `LAECHG` (the AECCARs a Bader analysis needs) and `LORBIT` $= 11$.

VASP writes its density as a `CHGCAR`, not as `.cube` files, and Calango registers it in the **Volumetric Data** dock when the run finishes — along with `AECCAR0`/`AECCAR2`, `LOCPOT` and `ELFCAR` when those were requested. That same `CHGCAR` is what an **Electronics** ⟩ **Electronic Structure** run reuses: with a VASP baseline selected it copies the file in and diagonalizes along the band path with `ICHARG` $= 11$, running no SCF of its own.

NCORE / KPAR Left at **auto** unless you know the machine: a wrong value is a performance cliff, not an error.

Extra INCAR tags One `TAG = value` per line, applied verbatim on top of everything above. No dialog can cover 300 INCAR flags, and one that tries becomes a ceiling.

> **Note**
>
> `ISMEAR` and `SIGMA` come from the shared **Smearing method** row rather than from this group, so one control drives every engine. Fermi-Dirac maps to `ISMEAR` $= -1$, Gaussian to 0 and Methfessel-Paxton to 1. **None** maps to a very narrow Gaussian rather than to the tetrahedron method `ISMEAR` $= -5$, which VASP refuses on the Γ-only meshes that small test cells use.
> `MAGMOM` is not set here either: it comes from the structure's initial magnetic moments, and ASE writes it out of the atoms object.

9.1.9 MACE models

MACE model **MACE-MP-0 (universal, materials)**, **MACE-OFF (universal, organic molecules)**, or **Custom trained model (file)**.

MACE model size `small`, `medium` (default), `large`.

Custom model file A `.model` or `.pt` checkpoint.

MACE device `cpu`, `cuda` or `mps`.

Foundation models download automatically on first use and are cached in `~/.cache/mace`.

9.2 The script editor

The right-hand pane always shows the complete, standalone ASE script the form describes — syntax-highlighted and fully editable.

Start typing in it and Calango shows *edited — form sync paused* in amber: from then on the form no longer overwrites your text, so hand edits are never silently lost. **Regenerate** discards them and re-syncs. **Save Script...** writes the current text to a `.py` file.

> **Tip**
> This is the intended escape hatch for anything the GUI does not expose. Configure what you can in the form, then edit the script for the rest — constraints, custom observers, a different optimiser. The script is ordinary ASE and runs unmodified on a cluster.

9.3 Execution environments

The **Execution Environment** group decides which Python actually runs the job — independent of the interpreter Calango embeds.

Leave it empty The job uses the embedded interpreter. The status line names it.

Env Folder... Point at a Conda environment directory; Calango finds `bin/python` inside it.

Python... Point directly at an interpreter.

The status line confirms the resolution live, turning red when no interpreter can be found at the given path. The setting is remembered between sessions, and the same selector appears in the phonon builder and the band-structure dialog.

> **Tip**
> This is how you keep heavyweight solver stacks isolated: a GPAW environment for band structures, a `mace-torch` environment with CUDA for ML potentials, and a minimal ASE environment for Calango itself.

9.4 The Job panel

Jobs run as separate processes in their own directory, so a crashing solver cannot take the GUI with it. The **Job** dock (zone 10) has five tabs.

9.4.1 Log

A status label (*Idle*, *Running…*, *Finished (exit N)*, *Crashed*), a progress bar, a **Kill** button, and the live output. Standard error is coloured red, the start line blue, and a clean exit green.

9.4.2 Metric plots

Four tabs plot job observables live, parsed from markers the generated scripts print:

Tab	Quantity	Unit	Reference line
Energy	Total energy	eV	—
Temperature	Ionic temperature	K	thermostat setpoint
Force	Max $\|F\|$	eV/Å	—
Pressure	Scalar pressure	GPa	barostat setpoint

Each plot shows the running series with the latest value read out numerically, and a dashed reference line at the setpoint for thermostatted or barostatted runs — so drift away from the target is visible at a glance. Every tab has its own **Export Data…** button writing CSV or whitespace-separated `.dat`. The series are saved in the project file, so a reopened project shows the plots from the run that produced it.

9.5 Live trajectory streaming

MD runs and geometry relaxations do not make you wait. The moment the job starts, Calango opens a trajectory tab marked *(live)* and streams each newly computed geometry into it as the simulation produces it.

- The timeline grows as frames arrive, and follows the newest frame automatically.
- Scrub back to an earlier frame and the view stays there — Calango stops following so you can inspect while the run continues.
- Everything else works normally on the partial trajectory: measurement, colour mapping, representation changes.
- When the job finishes the *(live)* marker is dropped and the tab becomes an ordinary trajectory.

Relaxations stream every step; MD streams at an interval chosen so a run produces at most about 400 streamed frames, keeping long simulations responsive. The complete trajectory is still written to `opt.traj` or `md.traj` in the job directory.

9.6 What happens when a job finishes

Calango inspects the job directory and does the useful thing:

- A live-streamed run keeps the tab it already built.
- An electronic-structure run opens the band/PDOS viewer (Chapter 11).
- A trajectory (`md.traj`, `opt.traj`) opens in a new tab.
- Otherwise, if a final geometry exists (`optimized.extxyz`), Calango offers to load it.

The process manager entry switches to *completed* or *failed*, and its directory link remains available for the rest of the session and beyond.

9.7 The MLIP dataset manager

Simulation ⟩ Dataset Manager (MLIP) assembles training datasets for machine-learning interatomic potentials from the trajectories you have generated — MD runs, noise ensembles, relaxation paths — and partitions them reproducibly.

9.7.1 Loading structures

Add Files... accepts multiple files of mixed provenance: `.extxyz`, `.traj`, `.cif`, VASP files, ASE databases. Each is listed with its frame count and the chemical formulas it contains, so a heterogeneous dataset stays legible. The summary line reports the total.

9.7.2 Splitting

Training, Validation Percentages; **Test** is the remainder, shown live.

Random seed Default 42.

The split is deterministic: the same frame count, percentages and seed always produce exactly the same partition, and the three subsets are disjoint and cover every frame.

9.7.3 Query-by-Committee ensembles

Active learning needs an ensemble of models trained on different data, so their disagreement can be used as an uncertainty estimate.

Committee members 1 exports a single dataset; $N > 1$ adds `committee_01...committee_N` subdirectories, each with its own training set.

Diversity How the members are made to differ:

- **Independent splits (seed + k)** re-splits the non-test pool with a different seed per member, keeping one common test set.
- **Bootstrap resampling of the training set** draws each member's training set with replacement — the classical bagging construction, which reuses roughly 63 % of the original frames per member.

9.7.4 Export

Extended XYZ (MACE-ready) Writes `train.extxyz`, `valid.extxyz` and `test.extxyz` (plus committee subdirectories), the layout MACE and most training scripts expect.

ASE database (.db) One SQLite database per subset.

> **Note**
> Export goes through `ase.io` directly rather than Calango's internal structure model, specifically so that energies, forces and stresses attached to the frames survive into the training files. A dataset exported from a finished MD run carries the labels a potential needs to train on.

CHAPTER 10

Remote HPC Execution

The **HPC** panel (zone 11) submits calculations to a cluster over SSH, monitors the queue, and brings the results back — without leaving Calango.

Requires
Needs `paramiko` in Calango's Python environment, and SSH access to a cluster running SLURM, PBS or SGE.

Note
All network traffic runs in a separate helper process driven over a JSON protocol, mirroring how local jobs are isolated: the interface never blocks on the network, and a hung connection can always be resolved by killing the helper rather than the application.

Note
The connection is a *session*, not a login per operation: Calango authenticates once and multiplexes every upload, submission, status poll, log tail and download over that one SSH transport. On a cluster with keyboard-interactive two-factor authentication this is the difference between usable and unusable — the one-time code is typed once, not once per queue poll.

10.1 Connection

The **Connection** tab holds the cluster credentials:

Host, Port Hostname and port (default 22).

User Your account name on the cluster.

Auth **SSH key** or **Password**.

Key file Path to a private key, e.g. `~/.ssh/id_rsa`. Leave empty to let your SSH agent or default keys authenticate.

Password The password, or the passphrase for an encrypted key.

Remote dir Working directory on the cluster, relative to `$HOME` (default `calango_jobs`).

Connect Opens the session, reports your remote `$HOME`, and auto-detects the scheduler. Pressing it first is optional: any operation issued while disconnected authenticates and then runs.

Disconnect Closes the session. A job already submitted keeps running on the cluster.

Two-factor authentication

If the cluster asks for a second factor, Calango puts the server's own challenge on screen — its wording, its prompts, with the answer masked whenever the server says it must not be echoed — and sends what you type straight back. Clusters that ask for the password and the code in one exchange get the password filled in from the **Password** field, so only the code is asked for.

The code is never stored, which is also why a session opened with 2FA is never re-established silently: if it drops, the panel says so and waits for you to press **Connect**. A session opened with a key or a password has nothing single-use about it and reconnects by itself on the next operation.

> **Requires**
> Passwords and key passphrases are kept in memory only — never written to disk, never placed on a command line where `ps` could see them. One-time codes are not kept even that long. Everything else (host, user, key path, scheduler settings) is remembered between sessions. A host key that *changes* is refused outright rather than pinned over.

10.2 Scheduler settings

The **Scheduler** tab describes the batch job:

Scheduler **SLURM**, **PBS** or **SGE**.

Queue Partition or queue name; empty uses the cluster default.

Tasks / cores Requested cores.

Walltime `HH:MM:SS`, default `01:00:00`.

Setup Shell lines executed before the payload — module loads, `conda activate`, anything your site needs:

```
module load python
source ~/venvs/ase/bin/activate
```

Submit Calculation... Opens the usual calculator dialog, then submits the result.

The generated wrapper carries the right directives for your scheduler — `#SBATCH`, `#PBS` or `#$` — and always redirects output to fixed file names so the monitor knows what to tail.

10.3 Submitting and monitoring

Simulation ⟩ **New Remote Calculation** (Ctrl+Shift+R) — or **Submit Calculation...** in the panel — runs the full sequence:

1. Stage `run.py` and `structure.extxyz` into a local job directory, exactly as a local run would.
2. Generate `job.sh` from the scheduler settings.
3. Upload the directory over SFTP, creating the remote path as needed.
4. Submit with `sbatch` or `qsub` and capture the job ID.
5. Start monitoring.

The **Queue & Logs** tab then shows the job ID and remote directory, the live queue state polled with `squeue` or `qstat`, and the remote standard output and error streamed incrementally into the console (errors in red). **Cancel Job** issues `scancel` or `qdel`.

10.4 Retrieving results

When the job leaves the queue, Calango downloads the results automatically — structures, trajectories, logs and CSV metric files — into the same local job directory the inputs came from. Trajectories and final geometries open directly in a new tab.

Download Results repeats the transfer manually, which is useful for pulling intermediate output from a long-running job without waiting for it to finish.

The task appears in the process manager like any other, so its directory stays one click away for later analysis.

CHAPTER 11

Electronic Structure

Simulation ⟩ Electronic Structure... computes a band structure along a high-symmetry k-path and, where the backend supports it, an element- and orbital-resolved projected density of states.

> **Requires**
> The GPAW route needs a **baseline ground state**: a completed **Simulation ⟩ Single-point Calculation...** run with the GPAW calculator, which writes `single_point.gpw` into its job directory. The bands run loads that converged density and evaluates non-self-consistently at fixed density — it never re-runs the SCF. Without a baseline the wizard refuses to open and tells you so. Chapter 2 walks through the whole sequence on silicon.

11.1 Setting up the calculation

Baseline SCF density The completed single point whose `single_point.gpw` supplies the charge density. Inheriting it rather than re-converging is what makes the band structure attributable to a specific, inspected ground state.

Backend **Free electrons (ASE — bands only)**, **GPAW (DFT, bands + PDOS)** or **Quantum ESPRESSO (needs pw.x + pseudos)**. The GPAW entry is disabled when the package is not importable in the selected environment.

k-path Pre-filled with ASE's suggestion for the structure's Bravais lattice, e.g. `GXWKGLUWLK,UX`. Edit it freely; leave it empty to accept the suggestion.

k-points Total points along the whole path, default 80.

Valence electrons Electrons per cell, free-electron backend only.

PW cutoff Plane-wave cutoff, default 340 eV.

SCF k-grid Monkhorst-Pack n^3 grid for the self-consistent step, default 4.

Compute element/orbital PDOS (GPAW backend) Adds the projected DOS.

Environment The Conda environment or interpreter that runs the job (Section 9.3).

> **Tip**
> The free-electron backend needs nothing beyond ASE and finishes in seconds. It computes empty-lattice bands — the exact free-electron dispersion folded into your Brillouin zone — which is genuinely useful for checking that a k-path is the one you meant, and for teaching

band folding, before committing a DFT run.

Requires

Real bands need a real solver: `gpaw` installed in the selected environment, or `pw.x` plus pseudopotentials for the Quantum ESPRESSO route. The QE script is a scaffold with an `EDIT ME` line for your pseudopotential set.

The job runs like any other — console, progress markers, process-manager entry — and writes `bands.json` and, when computed, `pdos.json` into the job directory. The viewer opens automatically on completion, and can be reopened any time with **Load Result** in the process manager.

11.2 Reading the plots

The viewer shows the band structure and, when present, the PDOS side by side on a shared energy axis.

11.2.1 Band structure

Energy versus k-path distance, with vertical lines and labels at every high-symmetry point (ASE's `G` is rendered as Γ). Spin channels are drawn in different colours. A dashed horizontal line marks the reference energy at $E - E_{\mathrm{ref}} = 0$.

11.2.2 Projected density of states

Density of states versus the same energy axis, one curve per element and orbital projection — `Si s`, `Si p`, `O p` and so on — accumulated over all atoms of each species and colour-keyed to the legend.

11.3 Controls and export

Fermi level The reference energy, pre-filled from the calculation. Plots show $E - E_{\mathrm{ref}}$, so shifting it moves the zero line — useful for aligning against a reference calculation or a measured spectrum.

E min, E max The energy window, defaults −10 and +10 eV.

Projections A checklist of the PDOS curves. Uncheck the ones that clutter the picture; the vertical scale re-normalises to what remains visible.

Export Bands... CSV or `.dat`: one row per k-point, with the path distance followed by every band (and spin channel).

Export PDOS... CSV or `.dat`: one row per energy, one column per projection.

Export Fatbands... CSV or `.dat`: one row per electronic state. Present only when the run wrote orbital projections — see §11.5.4.

> **Note**
> Exported bands use the same k-path distance as the x axis, so the columns drop straight into an external plotting script and reproduce the figure you see.

11.4 Band symmetry (irreducible representations)

A band structure says where the states are. Their *symmetry* says what they are: which crossings are protected, which optical transitions are allowed, and what a degeneracy is made of. Tick **Assign irreducible representations at high-symmetry k-points** in the setup stage and the run writes `band_symmetry.json` beside `bands.json`; the viewer draws the Mulliken symbol of every band beside its high-symmetry tick.

11.4.1 What is computed

At a wave vector $\mathbf{k}$ the operations $\{R \mid \mathbf{t}\}$ of the space group that satisfy $R^{-\mathsf{T}}\mathbf{k} = \mathbf{k} + \mathbf{G}$ form the *little group*. At a generic $\mathbf{k}$ that is only the identity; at the symmetry points of the zone it can be the whole point group, which is why the classification is worth making there and nowhere else.

For each degenerate multiplet the character

$$\chi(R) \;=\; \sum_{n \in \text{multiplet}} \langle \psi_{n\mathbf{k}} \mid \{R \mid \mathbf{t}\} \mid \psi_{n\mathbf{k}} \rangle$$

is evaluated in the plane-wave representation of the Kohn–Sham states. Writing $\psi = \sum_{\mathbf{G}} c_{\mathbf{G}} e^{2\pi i(\mathbf{k}+\mathbf{G})\cdot\mathbf{x}}$, the operation sends the coefficient at $\mathbf{G}$ to $\mathbf{G}' = \mathbf{G}_0 + R^{-\mathsf{T}}\mathbf{G}$ and multiplies it by $e^{-2\pi i(\mathbf{k}+\mathbf{G}')\cdot\mathbf{t}}$ — one permutation of the coefficient array plus a phase, exact on any grid and for any operation. The characters are then reduced against the character table of the little co-group, computed numerically from its own class-sum algebra, exactly as the Raman module does for phonons (§14.8).

The origin matters and is found rather than assumed. Moving the origin to $\mathbf{x}_0$ replaces every $\mathbf{t}$ by $\mathbf{t} - (1 - R)\mathbf{x}_0$, and the labels at a zone-boundary point are only convention-free once the origin sits at the symmetry centre. spglib reports the operations in whatever cell it was handed, which is routinely not that point — ASE's `graphene()` puts an atom at the origin, where the site symmetry is $\bar{6}m2$, while the $6/mmm$ centre is the hexagon at (1/3, 2/3).

11.4.2 Settings

Also classify the symmetry lines between them Classifies a generic point of each path segment as well as its endpoints. Together these are the *compatibility relations*: the irrep at a point decomposes into the irreps of the lines running out of it, which is how a degenerate level is seen to split and which branch goes where. One extra k-point per segment.

Symmetry tolerance spglib's `symprec` for the space-group operations. Too tight and a relaxed structure loses operations it physically has; too loose and it gains ones it does not.

Degeneracy window Bands closer than this share one character. It has to be a window rather than an exact test: two states degenerate *by symmetry* still emerge from a diagonalization a few μeV apart, and splitting them yields two meaningless one-dimensional characters instead of one correct two-dimensional one.

Energy window Only bands within this distance of E_F are classified. Each costs a transform per operation, and the states worth naming are the ones near the gap.

> **Tip**
> Graphene along Γ–K–M–Γ reproduces the classification of Kogan and Nazarov, *Phys. Rev. B* **85**, 115418 (2012): the two bands meeting at E_F at K form the two-dimensional E'' — that degeneracy *is* the Dirac point — with A_1' and E' below it, and A_{1g}, A_{2u} and E_{2g} among the occupied bands at Γ. This is the `graphene_band_symmetry` integration test.

> **Requires**
> At a zone-boundary point of a *nonsymmorphic* group the little-group representations are projective and no ordinary Mulliken symbol applies. The run detects this from the factor system, marks the point, and reports the raw characters rather than inventing a label. The viewer says so too.

> **Note**
> Unavailable together with spin-orbit coupling. The spinor bands are a different set of states from the scalar-relativistic ones, in a different number, and classifying them needs the double groups.

11.5 Orbital-projected bands (fatbands)

A PDOS says which orbitals contribute at a given *energy*. A fatband says which orbitals contribute to a given *band, at a given k-point* — the difference between "there is a band crossing E_F" and "the band crossing E_F is Fe d, so the magnetism lives there".

Tick **Compute orbital-projected bands (fatbands)** and the run carries the weight $|\langle\phi \mid \psi_{n\mathbf{k}}\rangle|^2$ of the selected orbitals alongside every band energy, writing `fatbands.json`.

11.5.1 Choosing the channels

Each row of the channel table is one projection:

Atoms An element symbol (`Fe`), a 0-based index list (`0, 2, 5-8`), or empty for every atom. Selecting individual indices is what distinguishes a surface layer from the bulk underneath it — the same element, a different question.

Orbital The shell (s, p, d, f) or one magnetic sub-level of it (p_z, d_{z^2}, …). Separating p_z from (p_x, p_y) is what separates the π bands of a layered material from its σ bands.

Label The name shown in the viewer; left blank it is generated from the selection.

Leave the table empty for one channel per element and per shell present in the structure — the useful default for a first look. **From structure** refills it with one channel per element on its valence shell.

11.5.2 Reading the plot

The viewer's **Orbital projections** panel lists the channels with their colours and offers three drawing modes:

Line width The classic fatband: the band thickens where the selected orbital contributes.

Colour scale Constant width, with the weight mapped onto the channel's colormap. Stays readable where many bands run close together.

Width + colour Both, and the default.

Several channels can be shown at once, which is the point: seeing metal d and ligand p on the same plot is how hybridization becomes visible. All channels share one normalization, so a channel contributing 2% of a state does not look as strong as one contributing 90%.

11.5.3 Colormaps and superposition

Each channel gets its own *sequential* colormap, assigned in order: Greens, Blues, Reds, Oranges, Greys, Purples — the ColorBrewer maps matplotlib ships under the same names, with one deliberate modification. **The alpha channel ramps linearly with the weight, so the lowest value is fully transparent rather than white.**

That single change is what makes overlapping projections readable. An opaque low end would paint over every channel already drawn *and* over the dispersion itself, so a six-orbital plot would show only whichever orbital happened to be drawn last. With a transparent floor each channel deposits ink only where it actually has weight, and the overlaps composite.

The plot background decides which way each ramp runs. A sequential colormap is a luminance ramp *away from the page*: on a white background it runs towards the saturated dark end, exactly as in matplotlib, and on the dark default plot background its lightness is mirrored so maximum weight is a bright hue instead of near-black. Hue and saturation are untouched, so Blues is blue either way. The channel list shows each channel's swatch, and its tooltip names the colormap — which is the name to put in a figure caption.

11.5.4 Exporting the weights

Export Fatbands... writes the weights themselves, as CSV or as a space-separated `.dat`. The table is *tidy*: one row per electronic state, not one row per k-point.

k_distance	spin	band	energy_eV	C_p_z	C_s
0.000	1	1	−20.14	0.01	0.83
0.000	1	2	−8.72	0.79	0.02

The shape is deliberate. A weight is uninterpretable without the energy of the state it belongs to, and keeping the two together in the wide layout the band export uses would take (channels × bands) columns. As written, the file drops straight into pandas or gnuplot as the (x, y, weight) triples a fatband plot consumes. Energies are absolute, matching **Export Bands...**, not shifted by the viewer's current Fermi reference; channel labels are underscored so a name like `C p_z` stays one column.

> **Note**
> GPAW backend only — the weights are the PAW projector overlaps, read through `gpaw.dos`, which the file-based DFT templates do not expose. Like the symmetry labels, unavailable together with spin-orbit coupling.

11.6 X-ray absorption spectroscopy (XAS)

Electronics ⟩ X-ray Absorption Spectroscopy (XAS)... computes a core-level absorption spectrum with GPAW, following the GPAW XAS tutorial.

An XAS transition starts in a *core* level of one atom, so it needs a PAW dataset with a hole in that level — and none ships with GPAW. The generated script therefore runs three stages rather than one: it builds the core-hole dataset for the absorbing element, converges a ground state that uses it on the absorbing *atom*, and evaluates the spectrum from those wavefunctions. The dataset is written into the job directory and found through `setup_paths`, so nothing is installed into your GPAW distribution and the run reproduces elsewhere.

11.6.1 Absorbing site

Pick the **Element** and then the **Absorbing atom** — one atom, not all atoms of that species. Giving the core-hole setup to every one of them would model a solid in which all are excited simultaneously, which is not what an absorption measurement does. Inequivalent sites each need their own run, and the measured spectrum is their sum; the wizard says so when the structure has more than one candidate.

Edge selects the level: K (1s) is what almost every XAS measurement means, while the L edges (2s, 2p) matter for transition metals.

11.6.2 Core hole

Half hole The transition-potential approximation — 0.5 electrons removed. One calculation gives the whole spectrum, because the final-state relaxation is averaged between initial and final states. This is the tutorial's choice and what most published XAS is.

Full hole The excited final state proper. Better for the first resonance, worse for the rest of the spectrum, and it needs the delta-Kohn-Sham shift to sit on an absolute energy scale.

No hole The unperturbed ground state — what the spectrum would be if the core hole did not pull the excited states down. Worth computing to see how large that effect is.

Unoccupied bands sets how many states above the occupied ones are converged. The spectrum is a sum over them, so too few truncates it — visibly, as a spectrum that stops rather than decays.

11.6.3 Broadening and the energy scale

Broadening (FWHM) is a uniform Gaussian on every transition. **Broaden more at higher energy** ramps it linearly above the edge, which is the physical behaviour (core-hole lifetime plus final-state broadening) and what makes the result comparable with a measurement whose peaks wash out with energy. The default ramp is the tutorial's, chosen for the oxygen K edge — move it to your own.

GPAW produces the spectrum on a *relative* scale whose zero is the first unoccupied state, which is not where a measurement puts it. **Compute the absolute edge position** adds two total-energy calculations (delta-Kohn-Sham) that give the shift, roughly doubling the cost; alternatively type a known **Edge energy** directly.

> **Note**
>
> `gpaw.xas` runs on GPAW's *legacy* engine only. This is the one script Calango generates that clears `GPAW_NEW` and asks for `legacy_gpaw=True` — without it the run stops with

"New-GPAW not supported".

11.6.4 Results

The viewer opens automatically when the run finishes and plots the isotropic spectrum — the average over the three Cartesian polarizations, which is what a powder or solution measurement sees. **Show the x, y and z polarizations** adds them individually; for an oriented sample the difference between them is the result. **Show individual transitions** overlays the discrete lines the broadened curve is built from, which is how you tell a genuine shoulder from two peaks the broadening merged.

The header states which energy scale the axis is on, because a relative spectrum plotted as though it were absolute is wrong by hundreds of eV and nothing about the curve reveals it.

11.7 Unfolding a relaxed supercell

Unfolding needs the supercell to be an exact integer multiple of the primitive cell: the projection is defined against that mapping. A *relaxed* doped supercell is not — substituting an atom and letting the cell relax moves its lattice constant by a per cent or so, and the pristine host is no longer $1/n$ of it.

The **Effective Bands** wizard handles that two ways.

Tolerance is how far M may sit from integers before the two cells are called incommensurate. The default (0.02) has room for a relaxation; an unrelated primitive cell misses by 0.1 or more, so the check still catches the mistake it exists for.

Force commensurability by rescaling the unit cell rebuilds the primitive cell as $P = M^{-1}S$, making the relation exact. For a relaxed supercell this is not a fudge: the supercell *is* $M\times$ something, just not $M\times$ the pristine host any more, and this recovers the host cell it actually relaxed into. Only the lattice is rebuilt — the basis rides along at the same fractional coordinates, so every atom stays on its site.

The wizard reports the resulting strain. A couple of per cent is the relaxation; much more means the wrong primitive cell was selected, and it says so.

> **Note**
> Worked example — Au_7Pt. A primitive Au fcc cell ($a = 4.078\,Å$) and a $2 \times 2 \times 2$ supercell with one Au replaced by Pt, relaxed with GPAW, comes back at $a = 4.1088\,Å$: a 0.755% expansion. The deduced $M = 2\mathbf{I}$ is right, but its residual is 0.015 — rejected outright by a pristine-cell tolerance, accepted by the default one. Forcing rebuilds the primitive at $4.1088\,Å$ and the residual falls to 5×10^{-17}.

CHAPTER 12

Optical and Quasiparticle Properties

Four workflows evaluate the response of a converged ground state: the dielectric function, the same response reduced to sheet quantities for a 2D material, the second-order (nonlinear) response, and the G_0W_0 quasiparticle correction.

Requires

The first two and G_0W_0 **inherit a baseline** rather than converging their own ground state — a completed GPAW **Simulation** ⟩ **Single-point Calculation…**, which writes `single_point.gpw`. They will not open without one.

This is not merely a saving. Re-converging inside the job would give a spectrum from a different SCF solution than the one you validated — different smearing, a different k-grid, possibly a different magnetic state — with nothing in the output to say so. Because the ground state is inherited whole, these wizards show no Calculator Settings stage: the cutoff, functional and k-grid all come back from the baseline, and offering to set them again would present knobs that cannot affect the run.

Nonlinear Optics (§12.3) is the exception, and for a reason worth knowing: `gpaw.nlopt` requires point-group symmetry to be off and its band sums need a *converged* empty manifold, neither of which a general baseline has. It therefore converges its own ground state and keeps a Calculator Settings stage.

12.1 Optical properties

Simulation ⟩ **Optics…** evaluates the frequency-dependent dielectric function $\varepsilon(\omega)$ with GPAW's linear-response module and derives the usual optical observables from it.

12.1.1 Settings

Baseline SCF (.gpw) The ground state to inherit. The note beneath it spells out what is being inherited — functional, cutoff, k-grid and environment — because with no Calculator Settings stage this is the only place those values are visible, and they are what decides whether the spectrum is trustworthy.

Broadening η Lorentzian broadening, default 0.1 eV. Smaller values resolve sharper features but need a denser k-mesh in the *baseline* to be meaningful.

Energy window min/max The photon-energy range, default 0–20 eV.

Tetrahedron integration See below.

Number of points Samples on the energy grid.

Polarization directions ε_{xx}, ε_{yy}, ε_{zz}. For an isotropic crystal the three coincide.

12.1.2 Brillouin-zone integration

By default the response is summed over k-points, giving every interband transition a Lorentzian of width η. Anything narrower than η — a van Hove singularity, a 2D absorption edge — is smeared into the broadening rather than resolved.

Tetrahedron integration interpolates the bands linearly within each tetrahedron and integrates analytically, resolving those features on a mesh where point integration is still noisy.

> **Requires**
> Tetrahedron integration requires the *baseline's* k-grid to contain every vertex of the irreducible Brillouin zone — a grid from `gpaw.bztools.find_high_symmetry_monkhorst_pack()`. An ordinary Monkhorst–Pack grid usually does not qualify, and since the grid is inherited, it has to be right when the *baseline* is run; the optics job cannot fix it afterwards. When the grid is unsuitable the run stops and says so, naming the remedy, rather than quietly falling back to point integration and returning a spectrum computed by a different method than the one requested. Which integrator ran is recorded in `optics.json` alongside the spectra.

12.1.3 The viewer

The results window plots one quantity at a time against photon energy: ε_1 and ε_2 together, the absorption coefficient $\alpha(\omega)$, reflectivity $R(\omega)$, the refractive index n and k, and the energy-loss function $L(\omega)$.

The **X axis** selector switches between **Energy (eV)** and **Wavelength (nm)** via $\lambda = hc/E$ with $hc = 1239.84197\,\text{eV nm}$. Every curve and every tick label follows.

> **Note**
> Samples at $E \leq 0$ have no finite wavelength and are omitted from the wavelength view — dropped from the abscissa and from every curve together, so nothing is shifted out of alignment. Since GPAW's energy grid starts at zero, expect the wavelength view to begin one sample in. Very low energies map to very long wavelengths, so a window that starts near 0 eV will stretch far into the infrared; set **Energy window min** above zero if that compresses the part of the spectrum you care about.

Export CSV... writes one row per sample with every quantity for the selected direction; **Export Image...** renders the current plot at high resolution.

12.2 2D optics

Modules ⟩ 2D Materials ⟩ 2D Optics... runs the same response and then divides the vacuum back out.

The reason is that a slab supercell's ε_{3D} is not a property of the material: double the vacuum padding and it moves, because the sheet's polarization is diluted over more cell. The wizard adds one question — **Vacuum axis**, seeded from the cell but worth confirming, since getting it wrong rescales every 2D quantity by the wrong length — and derives three quantities that do not depend on the padding:

Quantity	Definition	Units
Sheet polarizability	$\alpha_{2D} = \frac{L_z}{4\pi}(\varepsilon_{3D} - 1)$	Å
Absorbance	$A(\omega) = \frac{\omega L_z}{c}\,\mathrm{Im}[\varepsilon_{3D}]$	dimensionless
Sheet conductivity	$\sigma_{2D} = -i\,\omega\,\alpha_{2D}$	e^2/h

The -1 in α_{2D} removes the vacuum's own contribution, which is what makes the result a property of the sheet.

The viewer offers these three only when the file actually carries them, and a 2D job opens on the absorbance rather than on ε.

Tip

Graphene is the reference case. Its low-energy absorbance is the universal $A = \pi\alpha \approx 2.29\,\%$, fixed by the fine-structure constant alone, and its conductivity is the equally universal $\pi/2$ in units of e^2/h. If a graphene run reproduces those, the whole chain — baseline, response, vacuum-axis choice and unit conversion — is working. Reaching the plateau needs a dense mesh: it is the k-sampling near the Dirac point that decides this, not the cutoff or the band count.

12.3 Nonlinear optics

Electronics ⟩ Nonlinear Optics... evaluates the *second-order* response with GPAW's `gpaw.nlopt` module, within the independent-particle approximation. Three quantities are offered:

Second-harmonic generation $\chi^{(2)}(-2\omega;\omega,\omega)$: two photons of energy $\hbar\omega$ in, one of $2\hbar\omega$ out. Reported in pm/V, and for a sheet additionally as $\chi^{(2)}L$ in nm^2/V.

Shift current $\sigma^{(2)}(0;\omega,-\omega)$, in A/V^2: the DC photocurrent a homogeneous illuminated crystal carries with no junction and no built-in field — the bulk photovoltaic effect.

Linear susceptibility tensor $\chi^{(1)}(\omega)$, the full 3×3 tensor from the same matrix elements, and the $\varepsilon = 1 + \chi^{(1)}$ that follows from it.

Requires

$\chi^{(2)}$ and the shift current are **odd-rank tensors**: in a centrosymmetric crystal every component vanishes *identically*, by symmetry rather than by smallness. GPAW will still return numbers — the residue of an incomplete cancellation over a finite k-mesh — and they look like a spectrum. The generated script therefore tests the cell for an inversion centre *before* converging anything and warns, and the viewer repeats the warning above the plot. This is the single most common way the module is misused.

12.3.1 Why this one converges its own ground state

Unlike every other workflow in this chapter, Nonlinear Optics does **not** inherit a Single-Point baseline. Three requirements make an ordinary baseline unusable:

- `make_nlodata` asserts that *point-group symmetry is off*. The matrix elements it builds are not invariant under the point-group folding of the k-set, and the assertion arrives with no message — after the SCF has already been paid for.

- The sums run over *intermediate* states, so the band set has to be large. The wizard's ground state uses `nbands="nao"`.
- Those empty bands must be *converged*. An SCF converges occupied states; the unoccupied manifold it leaves behind is noise, and here it is summed over. The generated calculator asks for `convergence={"bands": -10}`.

All three are imposed by the generator rather than left on the Calculator Settings stage, because they are requirements of the method and not preferences. Time reversal is deliberately *kept*: the weaker `symmetry="off"` would satisfy the assertion too, at roughly double the k-points, for nothing.

12.3.2 Settings

Tensor components Three letters from `xyz`, several separated by commas (`yyy, xxy`). There is deliberately no "all 27": most vanish by symmetry, each costs a full sum over bands and k-points, and 27 spectra of which 24 are numerical noise is not a better answer. **Analysis** ⟩ **Symmetry** will name the point group that decides which are allowed.

SHG gauge **Length** (GPAW's default) and **Velocity** are formally equivalent and numerically are not: the velocity gauge carries low-frequency divergences that cancel only for a complete band set, so a truncated sum leaves it visibly wrong as $\omega \to 0$. Running both and overlaying them is the standard convergence test — their disagreement says more about the band summation than any single number does.

Broadening η A second-order spectrum is far more sensitive to it than a linear one: the resonances are sharper and the divergences at ω and 2ω are regularized by exactly this number, so a small η on a coarse mesh produces spikes rather than structure.

Scissors shift A rigid shift of the empty bands, for the semilocal gap error. $\chi^{(2)}$ is more sensitive to the gap than the linear response is, because the two-photon resonance sits at *half* of it. The applied value is recorded in the results and restated by the viewer, so a spectrum never silently carries one.

Vacuum axis As in 2D optics: a supercell $\chi^{(2)}$ is diluted by whatever vacuum was used, and multiplying the thickness back in gives the sheet value the 2D literature quotes.

> **Note**
> The expensive step is `make_nlodata` — the momentum matrix elements — and it runs *once*. Every component and every response reuses the same data, which is saved as `mml.npz` alongside the run. Asking for a second tensor component costs a band sum, not another ground state.

> **Tip**
> Remember that SHG probes the band structure at 2ω as well as at ω. A window ending at 6 eV is sampling transitions up to 12 eV, and the band count has to reach them — which is also why the $\chi^{(1)}$ panel is worth computing: the $\chi^{(2)}$ features of a semiconductor sit at the absorption edge *and* at half of it, and a resonance assigned to the wrong one of those is the standard way to misread an SHG spectrum.

12.4 GW calculations

Simulation ⟩ GW Calculations... computes one-shot G_0W_0 quasiparticle corrections,

$$E^{\mathrm{qp}}_{n\mathbf{k}} = \varepsilon^{\mathrm{DFT}}_{n\mathbf{k}} + Z_{n\mathbf{k}}\left[\Sigma_{n\mathbf{k}}(\varepsilon^{\mathrm{DFT}}) - V^{xc}_{n\mathbf{k}}\right],$$

i.e. the DFT eigenvalue with its exchange–correlation potential replaced by the energy-dependent, non-local self-energy. Nothing here is self-consistent, and the result is only meaningful relative to the baseline it corrects.

12.4.1 Engines

Two engines, each bound to the DFT code that produced its baseline. They are not interchangeable — G_0W_0 perturbs a specific DFT solution, so the engine follows from which baseline exists rather than from preference.

GPAW Corrects a `.gpw`. The run restarts the baseline, adds a generous set of empty bands at fixed density, then applies `gpaw.response.g0w0.G0W0`.

Yambo Corrects a Quantum ESPRESSO `.save` directory. The run converts it with `p2y`, generates the G_0W_0 input, executes it under MPI, and parses the quasiparticle report.

12.4.2 Settings

Screening cutoff The convergence parameter that matters. A G_0W_0 gap is not converged until it stops moving with this.

Frequency treatment **Plasmon-pole** approximates the frequency dependence of the screening with a single pole — much cheaper, and usually good for the gap of a simple semiconductor. **Real axis** performs the full integration.

Bands below/above How many occupied and empty states around the gap receive a correction.

MPI ranks Yambo only.

12.4.3 The viewer

The **GW Viewer** — also on **Results ⟩ GW Viewer...** — reports the DFT and quasiparticle band edges, the gap renormalization as its headline number, and a per-state table of $\varepsilon_{\mathrm{DFT}}$, E_{qp} and the correction between them.

> **Note**
> G_0W_0 essentially always *opens* a semiconductor gap, typically by 0.5–2 eV. A renormalization that is negative or near zero is far more likely an unconverged screening cutoff or too few empty bands than a physical result, and the viewer says so rather than presenting the number neutrally. The per-state corrections are the other thing to read: they should vary smoothly across bands, not jump erratically between neighbouring states.

CHAPTER 13

Electron–phonon coupling

Simulation ⟩ Electron-Phonon Coupling... computes the electron–phonon matrix elements by supercell finite differences through GPAW's `gpaw.elph` module, and reduces them to the two numbers that make the coupling usable downstream: the coupling constant λ and the electron–phonon relaxation time τ that the Drude term of the optics module (§12.1) takes as its input.

The module is GPAW-only, and locked rather than defaulted. Every other engine's ASE calculator exposes energies and forces; this method needs the change in the *effective potential* under an atomic displacement, which is not part of that interface.

13.1 The matrix elements

The electron–phonon vertex is the matrix element of the potential change produced by a phonon,

$$g^{\nu}_{mn}(\mathbf{k},\mathbf{q}) = \sqrt{\frac{\hbar}{2M\omega_{\mathbf{q}\nu}}}\,\langle m,\mathbf{k}{+}\mathbf{q} \mid \boldsymbol{\epsilon}_{\mathbf{q}\nu}\cdot\nabla V \mid n,\mathbf{k}\rangle, \tag{13.1}$$

where $\boldsymbol{\epsilon}_{\mathbf{q}\nu}$ is the eigenvector of mode ν at wavevector $\mathbf{q}$, M the ionic mass and the square root is the zero-point displacement amplitude. With that prefactor in place g carries units of energy; without it the bare element is in eV/Å and every quantity derived below is wrong by a mode-dependent factor. The generated script passes `prefactor=True` for exactly this reason.

∇V is not available analytically here: it is obtained by finite differences. Each atom of the primitive cell is displaced by $\pm\delta$ along x, y and z *inside a supercell*, and the resulting change in the self-consistent effective potential is cached. That is $6N + 1$ supercell SCF runs and is essentially the whole cost of the module; everything after it is minutes.

Two constraints follow from the construction and are checked before the expensive stage rather than after it:

1. $\partial V/\partial u$ is known only out to the supercell boundary, so the supercell sets the *range* of the interaction that can be represented — and it bounds which $\mathbf{q}$ exist at all. A $\mathbf{q}$ that is not a reciprocal lattice vector of the supercell has no dynamical matrix in the cache at any price.

2. The second delta function in (13.2) is evaluated at $\mathbf{k}{+}\mathbf{q}$, so the electronic mesh must be a multiple of the phonon mesh; otherwise $\mathbf{k}{+}\mathbf{q}$ is not a sampled state and has no eigenvalue.

The workflow runs in three stages: `DisplacementRunner` (the displacements), `Supercell.calculate_supercell` (projection of the potential gradients onto the LCAO basis) and `ElectronPhononMatrix.bloch_matrix` (rotation into the Bloch basis). The middle stage is why the ground state must be LCAO: it projects onto *basis functions*, and a plane-wave ground state has none to project onto.

13.2 The Eliashberg spectral function

Averaging $|g|^2$ over the Fermi surface and resolving it by phonon frequency gives the Eliashberg spectral function

$$\alpha^2F(\omega) = \frac{1}{N(E_\mathrm{F})} \sum_{\mathbf{q}\nu} \sum_{\mathbf{k}mn} \left|g^\nu_{mn}(\mathbf{k},\mathbf{q})\right|^2 \delta(\varepsilon_{n\mathbf{k}} - E_\mathrm{F})\, \delta(\varepsilon_{m\mathbf{k}+\mathbf{q}} - E_\mathrm{F})\, \delta(\omega - \omega_{\mathbf{q}\nu}), \tag{13.2}$$

with $N(E_\mathrm{F})$ the electronic density of states at the Fermi level, in the same per-spin per-cell normalisation as the sums. Both electron deltas sit *at* E_F: the scattering that matters for transport connects two states on the Fermi surface. This makes (13.2) a Fermi-surface integral, with the same slow k-convergence as the plasma frequency (§12.1), and it makes the quantity meaningless for a gapped system — the run detects a vanishing $N(E_\mathrm{F})$ and says so rather than returning a zero that looks like weak coupling.

Both delta functions are represented by Gaussians. The electronic width is not a cosmetic broadening: it *is* the Fermi-surface sampling, and trades directly against the k-mesh.

The first inverse moment of (13.2) is the mass-enhancement coupling constant,

$$\lambda = 2\int_0^\infty \frac{\alpha^2F(\omega)}{\omega}\,\mathrm{d}\omega, \tag{13.3}$$

and the logarithmic average frequency, reported alongside it because λ alone does not say whether the coupling is to soft or stiff modes, is

$$\omega_\mathrm{log} = \exp\left[\frac{2}{\lambda}\int_0^\infty \frac{\alpha^2F(\omega)}{\omega}\ln\omega\,\mathrm{d}\omega\right]. \tag{13.4}$$

13.3 From λ to the Drude relaxation time

In the high-temperature limit the phonon-limited scattering rate is Allen's result,

$$\frac{\hbar}{\tau} = 2\pi\lambda k_\mathrm{B}T, \qquad k_\mathrm{B}T \gg \hbar\omega_\mathrm{log}. \tag{13.5}$$

Two caveats are reported by the run itself, because each changes the number rather than merely qualifying it.

λ versus λ_tr. Equation (13.5) is written here with the mass-enhancement λ of (13.3), not the transport coupling λ_tr, which carries an extra $(1-\cos\theta_{\mathbf{k},\mathbf{k}+\mathbf{q}})$ backscattering weight inside the Fermi-surface average. For a free-electron-like metal such as aluminium or sodium the two agree to a few percent; for a transition metal with an anisotropic Fermi surface they need not.

The temperature range. Below roughly $\Theta_\mathrm{D}/3$ the true rate crosses over to the Bloch–Grüneisen T^5 behaviour and (13.5) *overestimates* it — so the reported τ is then a lower bound. The run compares the requested temperature against $\omega_\mathrm{log}/k_\mathrm{B}$ and warns when the limit does not apply.

13.3.1 The factor of two

The relaxation time is consumed by the Drude term of the optics module, and the two conventions involved differ by a factor of two. GPAW's `DielectricFunction` implements the free-carrier response as

$$\varepsilon_\mathrm{intra}(\omega) = 1 - \frac{\omega_p^2}{(\omega + i\,\mathrm{rate})^2}, \tag{13.6}$$

whereas the textbook Drude function is

$$\varepsilon_{\text{intra}}(\omega) = 1 - \frac{\omega_p^2}{\omega\,(\omega + i\Gamma)}, \qquad \Gamma = \frac{\hbar}{\tau}. \tag{13.7}$$

Expanding the denominator of (13.6) for rate $\ll \omega$ gives $(\omega + i\,\text{rate})^2 \simeq \omega\,(\omega + 2i\,\text{rate})$, so the two agree only if

$$\Gamma = 2\,\text{rate} \qquad \Longleftrightarrow \qquad \boxed{\text{rate} = \frac{\hbar}{2\tau}} \tag{13.8}$$

GPAW's own documentation notes the same discrepancy. A τ computed here and a `rate` quoted in a paper are therefore *not* interchangeable, which is why `epc.json` reports both τ and $\hbar/2\tau$, and why the Optics wizard takes τ in femtoseconds and applies (13.8) itself.

Together with the plasma frequency — which the k-point convergence module can measure directly — (13.8) fixes both parameters of the Drude model from first principles, with no fitting to the measured spectrum.

13.4 Settings

Supercell Repetitions the displacements are made in; the range of the interaction and the bound on the **q**-mesh.

Displacement δ Default 0.01 Å. Trades truncation error (anharmonicity contaminating the derivative) against numerical noise (the difference dominated by the SCF threshold).

LCAO basis `dzp` for production. Not a mode choice — the projection stage requires basis functions.

Electron k-mesh Ground state and Bloch rotation. Must be a multiple of the **q**-mesh.

Phonon q-mesh Bounded above by the supercell.

Phonon smearing Width of $\delta(\omega - \omega_{\mathbf{q}\nu})$, which bins the modes into $\alpha^2F(\omega)$ *for display only.* λ is summed over the modes exactly and does not change with it.

Temperature Where τ is evaluated. Temperature-dependent by physics, not by preference.

Coulomb μ^* Morel–Anderson pseudopotential for the superconducting T_c estimate. *Empirical* — nothing here computes it and nothing can. Conventionally 0.10–0.15 for sp metals, higher for transition metals.

> **Requires**
> T_c depends on μ^* *exponentially.* For aluminium $\mu^* = 0.10$ gives 1.8 K and 0.14 gives 0.7 K against a measured 1.18 K — a factor of 2.7 across a range that is entirely defensible. Quote a range over 0.10–0.15, never a single value.

There is deliberately *no* Fermi-surface smearing setting. The two $\delta(\varepsilon - E_F)$ factors are integrated with the linear tetrahedron method, which interpolates the bands inside each tetrahedron rather than requiring states to land near E_F, and so has no width to choose; the accuracy is set by the k-mesh alone. Earlier versions did expose one, and it was not cosmetic: on fcc aluminium at a 6^3 mesh λ ran from 0.009 to 31 as that width was varied by a factor of sixteen, with no plateau anywhere, so the reported number was whichever width happened to be configured. Removing the parameter, rather than documenting how to converge it, is what fixed that.

The wizard restates the cost live ($6N + 1$ runs on a supercell $n\times$ the cell) and refuses, naming the offending axis, when either commensurability condition fails.

13.5 Outputs

The GPAW run itself stops at the raw matrix elements, writing `elph_raw.txt` — a manifest naming the mesh — beside the eigenvalues, phonon frequencies, $\mathbf{k}+\mathbf{q}$ map and GPAW's own `gsqklnn.npy`. Calango then integrates them. The split exists because $|g|^2$ is (spins, q, k, modes, bands, bands) complex and reaches tens of gigabytes on a production mesh, so the analysis is meant to run *beside* the calculation rather than after a download. Inside Calango this happens automatically when the run finishes; on a cluster, run `calango-elph-analyze <directory>` in place.

The analysis writes `epc.json` carrying $\alpha^2F(\omega)$ on its frequency grid, λ, $\omega_{\log}$, $N(E_\mathrm{F})$, the relaxation time τ at the requested temperature, the rate $\hbar/\tau$, and the $\hbar/2\tau$ that GPAW's `rate` parameter takes. A count of imaginary modes is included: those are a real result — the structure is not at a local minimum — but α^2F integrates $1/\omega$ and cannot represent them, so they are excluded and the spectral function then describes only the stable modes.

13.6 Derived properties

Everything here comes from the same α^2F and costs nothing extra.

Superconducting T_c. The property α^2F was invented to predict. Calango evaluates the Allen–Dynes form

$$T_\mathrm{c} = f_1 f_2 \frac{\omega_{\log}}{1.2} \exp\left[\frac{-1.04(1+\lambda)}{\lambda - \mu^*(1+0.62\lambda)}\right], \tag{13.9}$$

with the strong-coupling correction f_1 and the spectral-shape correction f_2 (which needs the second moment $\bar{\omega}_2$, also reported). McMillan's $\theta_\mathrm{D}/1.45$ form is given beside it because the older literature quotes it, but $\omega_{\log}$ is a property of the computed spectrum whereas θ_D is a fit to a different measurement.

When $\lambda \le \mu^*(1+0.62\lambda)$ the formula has no solution: the screened repulsion beats the phonon attraction. That is reported as "not a phonon-mediated superconductor at this coupling" rather than as $T_\mathrm{c} = 0$, which would be indistinguishable from a converged calculation of a very small T_c.

Mode-resolved coupling and linewidths. $\lambda_{q\nu}$, normalised so that $N_q^{-1}\sum_{q\nu}\lambda_{q\nu} = \lambda$, and the phonon linewidth $\gamma_{q\nu} = \pi N(E_\mathrm{F})\omega_{q\nu}^2\lambda_{q\nu}$ that follows. A single λ cannot distinguish coupling concentrated in one soft branch from the same total spread across the spectrum, and $\gamma_{q\nu}$ is the one quantity in this module measurable by inelastic neutron scattering on the same material.

Mass enhancement. $m^*/m = 1+\lambda$, and with it the renormalisation of the linear specific-heat coefficient.

13.7 Validation

Validation runs the full workflow on fcc aluminium, whose coupling constant is well established at $\lambda \approx 0.4$; (13.5) then gives $\tau \approx 10\,\mathrm{fs}$ at room temperature, within a few femtoseconds of a Drude fit to the measured optical conductivity.

CHAPTER 14

The Analysis Toolbox

Every tool in this chapter opens from the **Analysis** menu. Most compute as soon as they open, run on a worker thread so the interface stays responsive, and export their data as CSV or whitespace-separated `.dat` — the format is chosen by the extension you type.

14.1 Radial distribution function

Analysis ⟩ Radial Distribution Function computes $g(r)$, the probability of finding an atom at distance r relative to a uniform density.

Element pair Two combos, each **Any** or a specific element. **Any**–**Any** gives the total RDF; a specific pair gives the partial.

r max Default 10 Å.

Bins Default 200.

Periodic boundary conditions Enabled and checked when the structure has a cell.

Frames For trajectories: **start**, **end** and **stride**, giving a frame-averaged $g(r)$ over any sub-range.

> **Note**
> With periodicity enabled, all periodic images within $r_{\max}$ are enumerated explicitly rather than relying on the minimum-image convention. The result is therefore valid beyond $L/2$ and correct for triclinic cells — both places where a naive implementation quietly produces artefacts.

14.2 Bond length and angle distributions

Analysis ⟩ Bond Length / Angle Distributions histograms either pair distances or three-body angles j–i–k within a cutoff.

Distribution **Bond length distribution** or **Bond angle distribution (j–i–k)**.

Species (pair / center–neighbor) Two element filters. For angles the first is the central atom.

Cutoff Default 3 Å.

Bins Default 90.

Angles are in degrees, lengths in Å; both handle periodic images exactly.

14.3 Coordination numbers (CN / GCN)

Analysis ⟩ Coordination Numbers (CN / GCN) computes per-atom coordination and generalised coordination numbers.

Neighbor cutoff **Covalent radii × tolerance** (default tolerance 1.15) or **Fixed cutoff radius** (default 3 Å).

Bulk CN reference The cn_{max} normalising the GCN — 12 for fcc/hcp, 8 for bcc, 4 for diamond, or **Auto (max CN found)**.

Results appear as a per-atom table (index, element, CN, GCN) with a summary line giving the ranges and means. Two buttons push the result onto the structure: **Color Viewport by CN** and **Color Viewport by GCN**.

> **Note**
> The generalised coordination number weights each neighbour by its own coordination, which distinguishes sites that plain CN cannot: a terrace atom, a step-edge atom and a vertex atom on a nanoparticle can share the same CN but have clearly different GCN. This is what makes GCN such a good descriptor for catalytic activity.
> Periodic images are enumerated explicitly, so a primitive fcc cell correctly reports CN = 12 even though all twelve neighbours are images of the single atom in the cell.

14.4 Static structure factor

Analysis ⟩ Structure Factor S(q) computes $S(q)$ from the (frame-averaged) pair distribution via a Lorch-windowed Fourier transform.

q range Defaults 0.3 to 12 $Å^{-1}$.

q points Default 400.

g(r) r max Upper bound of the Fourier integral, default 10 Å. Larger values improve low-q fidelity.

g(r) bins Default 400.

Frames Start/end/stride, as for the RDF.

14.5 X-ray diffraction

Analysis ⟩ X-Ray Diffraction (XRD) simulates a powder pattern with the Debye scattering equation.

Wavelength Presets for Cu Kα (1.54056 Å), Co Kα, Mo Kα, Cr Kα, or **Custom** to type your

own.

2θ range Defaults 10–90°.

Points Default 800.

Supercell repeat Periodic structures are repeated before the Debye sum, default 3. More repeats sharpen the Bragg peaks, but the cost grows as N^2 in atoms.

Unlike the other analysis dialogs, this one waits for you to press **Simulate**. The status line then reports which form factors were used: Waasmaier-Kirfel factors when every species is tabulated, otherwise a constant $f = Z$ approximation — in which case peak *positions* remain exact but high-angle *intensities* are approximate.

Two exports: **Export Curve…** writes the 2θ–intensity pattern, and **Export Peaks…** writes detected peaks (those above 2 % of the maximum) with their d-spacings from $d = \lambda/(2\sin\theta)$.

14.6 Warren-Cowley chemical order

Analysis ⟩ Warren-Cowley analysis quantifies chemical short-range order in multicomponent alloys through

$$\alpha_{ij} = 1 - \frac{p_{ij}}{c_j},$$

where p_{ij} is the probability that a neighbour of an i-type atom is of type j, and c_j is the overall concentration of j.

$\alpha = 0$ The ideal random alloy.

$\alpha < 0$ Ordering — unlike pairs preferred.

$\alpha > 0$ Clustering or segregation of like species.

Set **First shell cutoff** (default 3.2 Å) and, optionally, **Second shell cutoff** (default 4.8 Å). The dialog reports the concentrations, the mean coordination of each shell, and the full α_{ij} matrix for every ordered species pair, exportable as CSV.

> **Tip**
> A quick sanity check: a perfectly ordered B2 (CsCl-type) binary gives $\alpha_{AB} = -1$ in the first shell and $+1$ in the second, while a random decoration of the same lattice gives values near zero.

14.7 Local entropy

Analysis ⟩ Local Entropy Analysis computes the per-atom pair-entropy fingerprint of Piaggi and Parrinello, in units of k_B:

$$s_S^i = -2\pi\rho \int_0^{r_c} \left[\, g_i(r)\ln g_i(r) - g_i(r) + 1 \,\right] r^2 \, \mathrm{d}r,$$

with $g_i(r)$ the Gaussian-smoothed radial distribution around atom i.

Cutoff Integration cutoff, default 5 Å — three or four coordination shells works well.

Broadening σ Gaussian width, default 0.15 Å.

Radial grid points Default 100.

Average over neighbors Smooths s_i over each atom's neighbourhood, sharpening the contrast between ordered and disordered regions.

The result is stored as the per-atom field `local_entropy` and mapped onto the atoms immediately, with a distribution histogram in the dialog. *Lower* (more negative) values mark more ordered, crystal-like environments; higher values mark disordered, liquid-like ones. This makes it an effective order parameter for melting, solid-liquid interfaces and grain boundaries.

14.8 Raman modes

Analysis ⟩ Raman Modes performs a Γ-point factor-group analysis: which optical phonons are Raman-active, IR-active or silent, by symmetry alone.

The method uses no hard-coded character tables. spglib supplies the space group operations of the primitive cell; the character table of the isogonal point group is computed numerically from the class-sum algebra; the mechanical representation $\chi(R) = (\pm 1 + 2\cos\theta)\, N_{\text{unmoved}}(R)$ is reduced into irreducible representations; the acoustic translations are subtracted; and activity follows from the vector (IR) and symmetric polarizability (Raman) representations.

The dialog reports the space group, point group, number of atoms in the primitive cell, and a table of irreps with their degeneracy, optical mode count and activity.

> **Tip**
> For silicon in the diamond structure the result is the textbook $\Gamma = T_{2g} + T_{1u}$: one triply degenerate Raman-active optical mode, and the acoustic branch.

> **Note**
> Mulliken subscripts are assigned from Cartesian axis geometry. In low-symmetry orthorhombic groups the subscript convention can be permuted relative to a particular textbook's axis choice — the activities and degeneracies are unaffected.

> **Requires**
> Needs `spglib` and a periodic structure.

14.9 Magnetic space group

Analysis ⟩ Magnetic Space Group determines which of the 1651 magnetic space groups (MSGs) the structure realizes, from its coordinates *together with* its magnetic moments.

An ordinary space group asks only where the atoms are. Once the atoms carry moments, an operation that maps a moment onto its own reverse is no longer a symmetry — unless it is combined with *time reversal* T. Writing the full group as $\mathcal{M} = \mathcal{G} + \mathcal{A}$, with $\mathcal{G}$ the unitary

operations and $\mathcal{A}$ the antiunitary ones, the Belov–Neronova–Smirnova (BNS) classification has four types:

Type I $\mathcal{A}$ is empty. Every symmetry is unitary; time reversal is broken outright. 230 of them, one per space group.

Type II $\mathcal{A} = T\mathcal{G}$, the "grey" groups. T alone is a symmetry, which is only possible when every moment vanishes: these are the *non-magnetic* structures. 230 of them.

Type III $\mathcal{A} = Tg_0\mathcal{G}$ with g_0 a spatial operation that is *not* a pure translation. Half the parent group stays unitary; the rest survives only combined with T. 674 of them.

Type IV g_0 *is* a pure translation — an *anti-translation*. The magnetic unit cell is a supercell of the crystallographic one: the classic two-sublattice antiferromagnet. 517 of them.

The classification follows Watanabe, Po and Vishwanath, *Sci. Adv.* **4**, eaat8685 (2018).

14.9.1 Supplying the moments

Magnetic moments from loads whichever moments the structure carries — the converged `magmoms` of a finished calculation, or the `initial_magmoms` it was seeded with. Those answer different questions: the guess says what was *asked* for, the result says what the calculation *found*, and an ordering that collapsed during the SCF is exactly the case where they disagree.

Whichever is loaded, the moment table stays editable. Flip a sublattice, cant a moment out of the axis, or type in an ordering the file never carried, then press **Determine**. Asking "what would the magnetic space group be if this sublattice flipped?" is the normal way to use this module.

A collinear structure (all moments along z) is analysed as collinear; any transverse component switches to the non-collinear treatment, in which the moments are axial vectors that rotate with the operations.

14.9.2 Reading the result

BNS number The label $S.L$, e.g. `221.97`: S is one of the 230 ordinary space groups and L distinguishes its magnetic descendants. This is the identifier the tables are keyed by.

Type I–IV, with a one-line statement of what it means for the crystal, and — for type IV — the anti-translation itself.

Parent space group The S of the BNS label: the space group of the *unitary* subgroup.

Crystallographic space group What the same structure would be with the moments *ignored*. The comparison is the physics: magnetic order can only lower the symmetry, and the difference between these two is exactly what the magnetism broke.

Magnetic ordering Non-magnetic, ferromagnetic, ferrimagnetic or antiferromagnetic, read off $|\sum_i \mathbf{m}_i|$ against $\sum_i |\mathbf{m}_i|$, both reported.

Magnetic class In the moment table: the symmetry-equivalence class of each atom *under the magnetic group*. Two atoms of the same element in different classes are the two sublattices of an antiferromagnet.

The **Symmetry operations** tab lists every element of the group. Those marked $1'$ are the antiunitary ones — the spatial operation alone is not a symmetry, only its combination with time reversal is.

14.9.3 Tolerances

The positional tolerance is spglib's usual `symprec`. The moment tolerance is separate and in μ_B, because a moment converged to $1.98\,\mu_B$ against its neighbour's $-2.02\,\mu_B$ is one antiferromagnet, not two inequivalent sublattices, and only a tolerance in μ_B can say so. Raise it to absorb SCF noise; lower it to resolve a genuine ferrimagnetic inequivalence.

> **Tip**
> The same cell, three answers. Take Fe at $(0,0,0)$ and $(\frac{1}{2},\frac{1}{2},\frac{1}{2})$ in a cube — geometrically bcc, $Im\bar{3}m$ (229). Zero moments give the grey type II group. Parallel moments give type I, still with parent 229. Antiparallel moments give `221.97`, type IV: the body-centring translation now maps a moment onto its reverse, so it survives only with T, and the unitary group drops to the simple-cubic $Pm\bar{3}m$ (221). This is the `magnetic_space_group` integration test.

> **Requires**
> Needs `spglib` (with its magnetic tables) and a periodic structure. A structure carrying no moments at all is answered as a grey type II group, correctly — the dialog says so rather than leaving the reading unexplained.

14.10 Volumetric data

Analysis ⟩ **Volumetric Data** visualises 3D scalar fields: charge densities, electrostatic potentials, wavefunctions, ELF.

14.10.1 Loading fields

Load Field A... and **Load Field B...** read Gaussian `.cube` (including files written by Quantum ESPRESSO's `pp.x`), VASP `CHGCAR` / `LOCPOT` / `PARCHG` / `ELFCAR`, and XCrySDen `.xsf` grids. Each is reported with its grid dimensions and value range.

Field A defines the geometry; Field B is optional and colours it.

14.10.2 Render modes

Edit Volumetric Render offers two modes: **Isosurfaces** and **Color Slice**. Potential-map colouring used to be a third; it is now a group inside **Isosurfaces**, because it always was one — the same surface, extracted the same way at the same isovalue, differing only in what colours it.

14.10.3 Isosurfaces

The isovalue slider and numeric box re-extract the surface in real time. Extraction uses a marching-cubes-family algorithm on a tetrahedral decomposition of each grid cell, which is topologically unambiguous, with smooth normals from the field gradient and periodic stitching so surfaces close correctly across cell boundaries.

14.10.4 Potential map colour

The **Potential Map Color** group under **Isosurfaces** makes an EPM: the isosurface (say the electron density) shapes the geometry, and the field chosen under **Color by** (say the Hartree potential) is interpolated at every surface vertex and mapped through the **Color map**. **Invert palette** flips the ramp.

Custom range pins the colour scale to an explicit min/max instead of the colouring field's own extremes. A potential runs from deeply negative at the nuclei to nearly flat in the vacuum, so on the full range everything interesting sits in a sliver of the ramp — and a fixed window is also what lets two molecules be compared on one scale.

Because the colouring is an option on the surface rather than a mode, it applies to *every* ticked dataset at once: two orbitals can be shown side by side, both painted by the same potential.

> **Tip**
> The classic recipe: load the charge density as Field A, the local potential as Field B, pick an isovalue on the density that traces the molecular surface, and choose **Coolwarm** — the diverging palette puts the neutral potential at white, so electron-rich and electron-poor regions read immediately.

14.10.5 Slice planes

Color Slice cuts through the volume with a colour-mapped plane, oriented by Miller indices $(h\,k\,l)$ against the grid's own lattice and swept along its normal by the **Offset** slider.

Extent sets how far the plane is drawn, in unit cells across. **This unit cell only** keeps it inside the box the field was computed in — the honest extent, and the one to use when reading values off it. The replicated options tile the periodic field over the neighbouring cells, which is what makes a surface reconstruction or an adsorbate pattern legible instead of showing one cell floating on its own. **Outline the slice** draws its boundary, useful where the field has gone to zero and the quad would otherwise fade into the background.

Custom color range pins the ramp the same way the potential map's does: a few outlier voxels (a nuclear cusp, a spike at a boundary) otherwise compress everything else into one end of it.

14.10.6 Export

Export Isosurface (OBJ)... writes the triangle mesh with normals for external rendering; **Export Slice (CSV)...** writes the sampled plane as x, y, z, value.

14.11 Born charges and charged defects on VASP and Quantum ESPRESSO

Two modules that used to be GPAW-only now run on VASP and Quantum ESPRESSO as well, and in the Born case by a better route.

Born effective charges take the DFPT path on both: `LEPSILON = .TRUE.` for VASP, `ph.x` with `epsil = .true.` for QE. That is one linear-response run instead of $6N$ displaced SCFs, and it is an *analytic* derivative — there is no displacement amplitude to trade off against SCF noise and no linearity assumption left to check. The macroscopic dielectric tensor comes free with the same run. Both write the same `born_charges.json` the GPAW path writes, so the viewer and the phonon LO–TO block are unchanged.

> **Note**
> Both routes need an insulator or semiconductor. For a metal the macroscopic polarization is not defined: VASP writes no Born block and `ph.x` refuses `epsil`. The scripts say that rather than failing obscurely.

Charged defects read total energies and band edges from `vasprun.xml` or the `pw.x` output, and run one fixed-geometry SCF per charge state — `NELECT` for VASP (an *absolute* electron count, so $q = +1$ is one electron fewer) and `tot_charge` for QE (which follows the physical sign directly).

> **Requires**
> The FNV correction on these two engines is *delegated to pymatgen* (`pymatgen-analysis-defects`), reading `LOCPOT` for the potential alignment. Where pymatgen is not installed the run reports **uncorrected** energies and says so loudly. That is deliberate: uncorrected is a legitimate mode — it is exactly what a supercell-convergence study needs — whereas a hand-rolled correction that has never been checked against a reference is a plausible number of the right magnitude and the wrong value, applied silently to every point of the diagram.

14.12 Raman and infrared spectra

Electronics ⟩ Raman and IR Spectroscopy... computes both spectra of the Γ-point phonons. They describe the *same* modes and differ only in which electronic response couples to them:

Infrared intensity is the change in macroscopic *polarization* a mode produces, $\left|\sum_\kappa Z^*_\kappa \cdot e_\kappa / \sqrt{M_\kappa}\right|^2$. In a periodic crystal there is no molecular dipole to differentiate, so the Born effective charges Z^* are the only route to it.

Raman activity is the Placzek invariant $45a'^2 + 7\gamma'^2$ built from $\partial\chi/\partial Q$, the change in *polarizability* — the same response the Optics module evaluates, taken in the static limit.

Three engines are offered, and what one run of each can produce differs by orders of magnitude. This is the choice the wizard's cost estimate exists to inform, so it is worth stating plainly:

GPAW Finite displacements throughout: $6N$ force evaluations for the Hessian, and $6N$ further self-consistent runs (each followed by six dielectric evaluations) for $\partial\alpha/\partial u$. Z^* is *inherited* from a completed **Born Effective Charges** run rather than recomputed, because GPAW's own route to it is another $6N$ Berry-phase runs for a quantity the Raman spectrum never uses. Supplying one is therefore what turns the IR column on; without it the phonons and the Raman spectrum are computed as usual and every IR intensity is reported as zero, with `ir.computed = false` recording which happened.

VASP One DFPT run — `IBRION = 8` with `LEPSILON = .TRUE.` — returns the force constants, every ion's Z^* and ε_∞ together, so the *infrared* spectrum costs a single job and nothing has to be selected for it. The Raman half has no such shortcut: VASP computes no Raman tensor, so $\partial\alpha/\partial u$ comes from differencing ε_∞ over $6N$ displaced `LEPSILON` runs.

Quantum ESPRESSO One `ph.x` run at $q = 0$ returns all of it. `epsil` gives the force constants, Z^* and ε_∞; `lraman` adds the Raman tensor as an analytic *third-order* response — no displacement amplitude to trade off against SCF noise, and no linearity assumption left to check.

> **Requires**
> `lraman` is implemented for **norm-conserving** pseudopotentials only. With an ultrasoft

or PAW set `ph.x` declines rather than approximating, and the run stops with a message naming that as the cause. The infrared half is unaffected — turn the Raman spectrum off and the same job still produces it. Edit the pseudopotential map in the generated script before running: the guess is the conventional `<Symbol>.UPF` naming, which a real library rarely matches.

All three engines share one physics core — the mass-weighted diagonalization, the two contractions, the Stokes prefactor and the broadening — and write the same `raman_ir.json`, so one viewer serves all of them. The file records which route produced it: `engine` and `method` name the workflow, and `displacement_A` is 0 for a DFPT result, because there was no displacement to report.

Note

The laser wavelength, temperature, broadening and frequency window change only the *reported* spectrum, not the physics. Mode frequencies and activities are computed once and the experiment-specific factors applied afterwards, so trying a different laser line does not mean another run — reopen the viewer instead.

14.13 Partial charges

Analysis ⟩ **Partial Charge Analysis...** partitions a converged electron density among the atoms. Three schemes are offered — **Bader** (topological zero-flux basins), **Voronoi** (nearest-atom cells) and **Hirshfeld** (stockholder weights against a promolecule) — and all three are native: they run on a standardized 3D grid and know nothing about the code that produced it.

14.13.1 Density source and engine

Charge-density source lists the completed processes whose directories hold a density; **Density format** decides how it is read. `Auto-detect` covers the normal case and the explicit entries exist for a directory holding output from more than one engine, where guessing would be a coin toss.

GPAW `get_all_electron_density()` from the run's `.gpw`.

VASP `AECCAR0` + `AECCAR2` when the run wrote them (`LAECHG = .TRUE.`), otherwise `CHGCAR`.

Quantum ESPRESSO `pp.x` (`plot_num = 0`) exports the density to a cube first. Set `CALANGO_PP_X` if `pp.x` is not on `PATH`.

Requires

Bader needs the all-electron density. Partitioned on the pseudo-valence density alone, its basins follow the wrong topology and the charges come out systematically small. For VASP that means `AECCAR0` and `AECCAR2`; given only a `CHGCAR` the run says so in its log rather than returning quiet nonsense.

14.13.2 Scope

Current structure only partitions the displayed frame. **All structures in the trajectory** partitions every frame — and it is not a loop over one density. A charge is a property of a *converged density*, so each frame needs its own; the run looks for one density file per frame and reports what it found rather than reusing a single density as though it described every geometry.

14.13.3 Reading and keeping the results

Load Results... fills the per-atom table and a summary line above it: the net charge, the range, and the mean per element — the number people actually quote, and the one a 200-row table does not give up easily.

> **Tip**
> The net charge is the check to read first. For a neutral cell it must come out near zero; anything larger is density the integration lost, and it bounds how much any single per-atom value can be trusted. The summary flags a residual above 0.05 e on its own.

Write into Trajectory stores the charges on the structure as `initial_charges` — the array name ASE reads and writes — through the document's undo stack. Saving as extended XYZ then carries them as a column of the file, and they appear in **Edit Structure...** (§7.2.2). The same field tints the viewport, so the colours and the file cannot disagree.

14.14 Charge density difference

Analysis ⟩ Charge Density Difference (CDD)... computes

$$\Delta\rho \;=\; \rho_{A+B} \;-\; \rho_{A} \;-\; \rho_{B},$$

the redistribution of charge caused by putting two fragments together. Each of the three densities on its own is dominated by the atomic cores and shows nothing; the difference shows the bond.

14.14.1 Step 1 — density source

Pick a completed **Simulation ⟩ Single-point Calculation**; that run supplies ρ_{A+B} and, more importantly, the calculator both fragments are rebuilt from. Choose whether to difference the **All-electron density** or the **Pseudodensity**. The pseudodensity is usually the clearer field to read: it carries no nuclear cusps, so the isosurface scale is set by the bonding features rather than by the cores.

GPAW, VASP and Quantum ESPRESSO are all supported, and the engine is taken from the parent run's provenance rather than asked for again — $\Delta\rho$ only means something if all three densities were computed the same way.

GPAW Restarts from the `.gpw` and reads the exact calculator back out of it. This is the strongest of the three guarantees: there is nothing left to re-specify.

VASP Reads `CHGCAR`, or `AECCAR0` + `AECCAR2` for the all-electron form (which needs `LAECHG = .TRUE.` on the parent run). Each fragment is re-run with the parent's settings.

Quantum ESPRESSO Exports each density through `pp.x` (`plot_num = 0`) as a Gaussian cube, since QE keeps its density in a binary file with no public reader. Set `CALANGO_PP_X` if `pp.x` is not on `PATH`.

> **Requires**
> For VASP and Quantum ESPRESSO the fragment runs pin the FFT grid explicitly to the parent's (`NGXF`/`NGYF`/`NGZF`, `nr1`/`nr2`/`nr3`). Both codes choose that grid from the cutoff *and* the cell contents, so a fragment with fewer atoms in it can otherwise land on a different one — and two densities sampled on different grids cannot be subtracted at all. GPAW needs

none of this because the restart already fixes the grid.

14.14.2 Step 2 — subsystem partition

Two columns list the atoms of the selected run. Move atoms across with the arrow buttons or by double-clicking a row. The split is exhaustive — every atom belongs to exactly one side — because $\Delta\rho$ only integrates to zero if A and B together are the whole system.

14.14.3 What the run does

Both fragments are recomputed in the parent's own unit cell, at the parent's geometry, with the parameters read back out of its `.gpw`: cutoff, XC functional, grid, k-points and convergence cannot drift between the three terms. *Nothing is relaxed.* $\Delta\rho$ is defined at one geometry, and letting a fragment relax would mix charge transfer with structural rearrangement into a quantity nobody can read off an isosurface.

> **Note**
> Cutting a closed-shell molecule in half leaves open-shell fragments, and an isolated atom with a partly-filled degenerate shell often will not converge under the settings that converged the molecule in one pass. The run therefore tries the parent's exact parameters first, then adds occupation smearing, then spin polarization, and reports in the log which was needed. The reported `net` charge is the check: A and B together hold exactly the electrons $A{+}B$ does, so it must come out at zero.

14.14.4 Result

The difference is written as `cdd.cube` and loaded automatically into the **Volumetric Data** dock (§14.10), where it renders in the main 3D viewport like any other grid. A **Coolwarm** colormap is the natural choice: accumulation and depletion sit on opposite sides of white. The status bar reports how much charge was redistributed, from `cdd.json`.

14.15 Adsorption and catalysis

Analysis ⟩ **Adsorption & Catalysis** finds high-symmetry adsorption sites on a slab and populates them.

14.15.1 Site detection

Opening the dialog detects sites on the top surface automatically and reports a census, for example *4 top, 12 bridge, 4 fcc, 4 hcp*:

`top` Directly above a surface atom.

`bridge` Midway between nearest-neighbour surface atoms.

`fcc`, `hcp` Threefold hollows, distinguished by whether a second-layer atom sits directly underneath (hcp) or not (fcc).

`hollow` A threefold hollow that could not be classified, typically because there is no second layer.

Neighbour vectors are enumerated over periodic images, so the census is correct even for a small $p(1 \times 1)$ cell where surface atoms bond to their own periodic copies. The table lists every site

with its in-plane coordinates and can be filtered by type.

14.15.2 Placing adsorbates

Adsorbate `OH`, `O`, `H`, `CO`, `CHO`, `H2O`, `N2`, `O2`, `NH3`, `CH4`, or any `ase.build.molecule` name or chemical formula you type.

Height Anchor-atom height above the surface layer, default 2 Å.

Place on Selected Sites Adds one copy per selected table row.

Molecules are anchored by the chemically sensible atom and oriented upright: O down for `OH` and `H2O`, C down for `CO` and `CHO`, N down for `NH3`.

14.15.3 Coverage series

The **Coverage series** group generates a systematic set in one step: choose a **Site family** and check the coverages you want — 0.25, 0.50, 0.75, 1.00 ML. Calango places evenly strided subsets of that site family and opens one labelled tab per coverage, ready for a batch of adsorption energy calculations.

14.16 Brillouin zone and k-path builder

Analysis ⟩ **Brillouin Zone / k-Path** shows the first Brillouin zone as a Wigner-Seitz cell of the reciprocal lattice, with high-symmetry points labelled from ASE's Bravais lattice detection.

Drag to rotate, Shift+drag to pan, wheel to zoom; **Orthographic projection** removes perspective foreshortening when you need to read the zone geometry.

14.16.1 Building a path

Click high-symmetry points in the 3D view to append them to the path. The list shows the sequence with fractional coordinates.

Suggested Loads ASE's suggested path for the lattice.

Break Starts a new discontinuous section, giving paths like $\Gamma \to X \,|\, M \to R$.

Undo, Clear Remove the last point, or start over.

Points per segment Sampling density, default 40.

14.16.2 Export

Export k-Path... writes the path for an external code:

Format	Typical file
VASP `KPOINTS` (line mode)	`KPOINTS`
Quantum ESPRESSO `K_POINTS crystal_b`	`kpath_qe.in`
CASTEP `SPECTRAL_KPOINT_PATH`	`kpath_castep.cell`
SIESTA `BandLines`	`kpath_siesta.fdf`
ASE / Python script	`kpath_ase.py`

Export Figure (PNG/SVG)... renders the zone at high resolution; the SVG output is vector and drops straight into a paper figure.

CHAPTER 15

Publication Output

15.1 Still images

File ⟩ **Import / Export** ⟩ **Export Image** (Ctrl+E) renders the current view off-screen at any resolution, independent of the window size, with 8× multisample anti-aliasing.

Resolution preset 720p, 1080p, 4K, or custom width and height up to 8192 px.

Background **Transparent (PNG only)**, **Solid white**, or a custom colour.

Format PNG or JPEG, chosen by the extension.

> **Tip**
> Transparent PNG is the right choice for figures that will sit on a coloured background in a paper or slide. Combine it with an orthographic, axis aligned view (O, then an alignment button) for a clean, reproducible result.

15.2 Animations

File ⟩ **Import / Export** ⟩ **Export Animation (GIF/MP4)** writes either a turntable rotation of a static structure or a playback of a trajectory.

Source **Turntable rotation (360°)** or the current trajectory.

Resolution preset Up to 4K.

Frames per second Playback rate.

Background Including **Transparent (GIF only)**.

Format Animated GIF or H.264 MP4.

> **Requires**
> GIF export needs `pillow`; MP4 needs `imageio` and `imageio-ffmpeg`. MP4 has no alpha channel — asking for a transparent MP4 falls back to a solid background.

15.3 Ray-traced rendering

File ⟩ Import / Export ⟩ Ray-Traced Render exports the active scene — camera, lights, representation, colours, multiple bonds, unit cell — to an external ray tracer for publication-quality output with true shadows and reflections.

Engine **POV-Ray** or **Tachyon**.

Width (px), Height (px) Default 1920 × 1440.

Background **Solid white** or **Viewport color**.

Renderer binary Path to the executable; the bare names `povray` and `tachyon` are resolved on `PATH`. The path is remembered per engine.

Two ways to use it:

Save Scene File... Writes just the scene (`.pov` or `.dat`) for you to render elsewhere, or to edit by hand before rendering.

Render... Writes the scene next to your chosen PNG output and invokes the renderer, streaming its output into the log pane so you can watch progress and diagnose failures.

> **Requires**
> Needs POV-Ray or Tachyon installed separately. If the binary cannot be started, the log says so explicitly rather than failing silently.

15.4 Data exports at a glance

Nearly every analysis tool exports its numbers, so figures can be regenerated in your own plotting stack.

Source	Formats	Typical file
RDF	CSV, DAT	`rdf.csv`
Bond length / angle	CSV, DAT	`bond_lengths.csv`
Structure factor	CSV, DAT	`structure_factor.csv`
XRD pattern	CSV, DAT	`xrd_pattern.csv`
XRD peak list	CSV, DAT	`xrd_peaks.csv`
Warren-Cowley	CSV	`warren_cowley.csv`
Job metric series	CSV, DAT	`energy.csv` ...
Band structure	CSV, DAT	`bands.csv`
PDOS	CSV, DAT	`pdos.csv`
Isosurface mesh	OBJ	`isosurface.obj`
Volumetric slice	CSV	`slice.csv`
Phonon bands / DOS	CSV	`phonon_bands.csv`
k-path	5 codes	Section 14.16
ML datasets	extxyz, ASE db	Section 9.7
Brillouin zone figure	PNG, SVG	`brillouin_zone.svg`

CHAPTER 16

Reference

16.1 Keyboard shortcuts

16.1.1 Viewport

Single-letter shortcuts, no modifier. They yield to text fields while you are typing.

Key	Action
R	Rotation mode
T	Translation (pan) mode
S	Selection mode
I	Insertion mode
D	Distance measurement mode
A	Angle measurement mode
O	Toggle orthographic / perspective
F	Frame structure
Delete / Backspace	Delete selection (Selection mode)

16.1.2 Application

Shortcut	Action
Ctrl+N	New workspace
Ctrl+O	Open structure
Ctrl+T	Open trajectory
Ctrl+Shift+O	Open project
Ctrl+S	Save project
Ctrl+Shift+S	Save structure as
Ctrl+Shift+T	Save trajectory as
Ctrl+E	Export image
Ctrl+W	Close tab
Ctrl+Q	Quit
Ctrl+Z	Undo
Ctrl+Shift+Z	Redo
Ctrl+Shift+A	Add atom
Ctrl+B	Bond editor
Ctrl+R	New calculation
Ctrl+Shift+R	New remote calculation

On macOS, Ctrl in this table is Cmd.

16.1.3 Mouse

Gesture	Action
Left drag	Depends on interaction mode
Middle drag	Pan (any mode)
Shift+left drag	Pan (any mode)
Wheel	Zoom
Left click	Pick atom
Ctrl/Cmd+click	Toggle atom in selection
Double click	Frame structure

16.2 Menu map

File New Workspace · Open (Structure, Trajectory) · Save (Structure As, Trajectory As) · Import / Export (Image, Animation, Ray-Traced Render) · Project Workspace (Open, Save, Save As) · Close Tab · Quit

Edit Undo · Redo · Add Atom · Selection (Change Element, Translate) · Delete Selected Atoms · Bond Editor · Unit Cell (Create Supercell) · Preferences

View Orthographic · Overlays (Show Unit Cell) · Alignment (Frame Structure, XY, XZ, YZ) · Visual Effects · panel toggles

Build Nanomaterial Builder · Surface Slab · Metallic Nanoparticle (Wulff) · Special Quasirandom Structure (SQS) · Normal Modes / Phonon Builder · From Database · Structure Perturbation / Noise

Simulation New Calculation (ASE) · New Remote Calculation · Electronic Bands / PDOS · Dataset Manager (MLIP)

Analysis Structure Factor S(q) · X-Ray Diffraction · Radial Distribution Function · Bond Length / Angle Distributions · Coordination Numbers (CN / GCN) · Warren-Cowley analysis · Local Entropy Analysis · Raman Modes · Volumetric Data · Adsorption & Catalysis · Brillouin Zone / k-Path

Help Documentation · GitHub Repository · About Calango

16.3 Python dependencies by feature

Package	Needed for
`ase`, `numpy`	Everything (structure I/O, building, jobs)
`spglib`	Symmetry detection, Raman modes
`paramiko`	HPC panel
`pillow`	GIF animation export
`imageio`, `imageio-ffmpeg`	MP4 animation export
`mace-torch`	MACE machine-learning potentials
`gpaw`	DFT band structures and PDOS
`icet`	Preferred SQS backend (optional)

External binaries: `pw.x` (Quantum ESPRESSO), `siesta`, VASP, `orca`, `povray` or `tachyon`.

16.4 Job directory contents

A typical job directory under `.calango_tmp/` (Section 3.4.1):

File	Written by
`run.py`	The generated (possibly hand-edited) script
`structure.extxyz`	The staged input geometry
`opt.traj`	Geometry optimization trajectory
`md.traj`	Molecular dynamics trajectory
`optimized.extxyz`	Final relaxed geometry
`bands.json`, `pdos.json`	Electronic structure results
`phonon_bands.csv`, `phonon_dos.csv`	Phonon results
`perturbed.extxyz`	Noise ensemble checkpoint
`job.sh`	Scheduler wrapper (remote jobs)
`calango_job.out`, `calango_job.err`	Remote stdout/stderr

16.5 Progress markers

Generated scripts communicate with the interface by printing markers to standard output. If you hand-edit a script or write your own, emitting these keeps the Job panel live:

Marker	Effect
`CALANGO_PROGRESS step total`	Progress bar
`CALANGO_ENERGY step value`	Energy plot (eV)
`CALANGO_TEMP step value`	Temperature plot (K)
`CALANGO_FMAX step value`	Force plot (eV/Å)
`CALANGO_PRESSURE step value`	Pressure plot (GPa)
`CALANGO_TARGET_TEMP value`	Thermostat reference line
`CALANGO_TARGET_PRESSURE value`	Barostat reference line
`CALANGO_CELL` + `CALANGO_FRAME`	Live trajectory streaming
`CALANGO_RESULT ...`	Final summary line

16.6 Troubleshooting

Structures will not open; job features are disabled. ASE is not importable. Run `calango --probe-python` to see which interpreter is being used, then either install ASE there or set `CALANGO_PYTHON` to an environment that has it (Section 1.2).

The space group is `P1` for an obviously symmetric crystal. Numerical noise in the coordinates. Raise **Tolerance** in the **Structure** panel — a relaxed cell often needs 0.01 Å or more.

Bonds are missing or spurious. Adjust **Covalent cutoff multiplier** in the bond editor (Section 7.4), or add and suppress individual bonds by hand. Metallic and ionic systems rarely match a covalent-radius rule.

Double and triple bonds do not appear automatically. By design. Assign them with the **Bond order** buttons (Section 7.5).

A job fails immediately with an import error. The job is running in a different environment from Calango itself. Check the **Execution Environment** status line in the calculation dialog (Section 9.3).

The GPAW backend is greyed out. GPAW is not importable in the selected environment. Point the **Environment** selector at a Conda environment where it is installed.

The remote panel reports a connection failure. Test the same host, port, user and key with `ssh` from a terminal first. Remember that the passphrase field doubles as the key passphrase in **SSH key** mode.

Adsorption site detection finds no bridge or hollow sites. The surface cell is too small to have in-plane neighbours. Build a supercell of the slab first.

The viewport looks soft after enabling depth of field. Expected — the effect trades multisample anti-aliasing for the blur pass (Section 4.7). Disable it before exporting still images.

Animation export fails. Install `pillow` for GIF, or `imageio` and `imageio-ffmpeg` for MP4, in Calango's Python environment.

16.7 Further reading

`README.md` Feature overview and build instructions.

`docs/packaging.pdf` Building the macOS and Linux installers.

`https://github.com/seixas-research/calango` Source, issues and releases.

`https://wiki.fysik.dtu.dk/ase/` ASE documentation — the reference for anything in a generated script.